PROGRAMA ACADÉMICO TEÓRICO–PRÁCTICO (36 HORAS)

Aprendizaje por Refuerzo

De la Probabilidad a las Redes Neuronales Profundas



Profesor Especialista en Aprendizaje por Refuerzo

Metodología: Lápiz, Papel y Código · Caso Conductor: OptiBot

Edición 2026

Prólogo: Fundamentos del Aprendizaje por Refuerzo

El aprendizaje por refuerzo (*Reinforcement Learning*, o RL) constituye uno de los paradigmas más representativos y potentes de la inteligencia artificial contemporánea. A diferencia del aprendizaje supervisado clásico, en el cual los modelos dependen de conjuntos de datos previamente etiquetados por un instructor externo, el aprendizaje por refuerzo modela el proceso mediante el cual un agente autónomo aprende a tomar decisiones óptimas interactuando directamente con su entorno mediante ensayo, error y señales escalares de recompensa.

Alcance y Estructura del Texto

Este texto ha sido estructurado con el objetivo de ofrecer una exposición progresiva, autosuficiente y matemáticamente transparente para estudiantes e ingenieros que posean conocimientos elementales de programación en Python y álgebra básica.

No se asumen conocimientos avanzados previos en probabilidad axiomática, procesos estocásticos ni teoría de control moderno. Cada concepto fundamental se introduce a partir de sus principios elementales, vinculando tres componentes en cada etapa:

1. **Intuición conceptual:** El planteamiento físico o lógico del problema de toma de decisiones.
2. **Deducción y cálculo manual paso a paso:** El desarrollo explícito de los cálculos numéricos y algebraicos en casos reducidos, permitiendo comprender la mecánica exacta de las ecuaciones.
3. **Implementación algorítmica:** La traducción directa de las fórmulas matemáticas a implementaciones ejecutables y reproducibles en Python (NumPy y librerías de aprendizaje profundo).

El Hilo Conductor: El Sistema Logístico OptiBot

Con el fin de mantener una coherencia conceptual continua a lo largo de toda la obra, el desarrollo teórico se articula en torno a un caso de aplicación industrial representativo: **OptiBot**, un robot móvil autónomo destinado a la gestión de inventario y transporte de carga en un almacén inteligente.

- Los Capítulos 1 y 2 abordan la cuantificación de la incertidumbre estocástica y la toma de decisiones evaluativas bajo el dilema de exploración frente a explotación (Bandidos Multi-Brazo).
- Los Capítulos 3 y 4 formalizan la evolución dinámica del entorno mediante Cadenas de Markov y Procesos de Decisión de Markov (MDP), deduciendo analíticamente las Ecuaciones de Bellman bajo modelos analíticos conocidos.

- El Capítulo 5 desarrolla el **Aprendizaje por Refuerzo Tradicional**, presentando el salto al aprendizaje libre de modelo mediante Diferencia Temporal y deduciendo exhaustivamente el algoritmo tabular **Q-Learning**.
- El Capítulo 6 culmina con el **Aprendizaje por Refuerzo Profundo**, abordando la transición hacia espacios sensoriales continuos mediante Redes Neuronales Profundas y el algoritmo **Deep Q-Learning (DQN)**.

El texto invita al estudiante a realizar activamente los cálculos manuales y experimentos computacionales propuestos en cada sección, consolidando una comprensión profunda y operativa del aprendizaje por refuerzo.

Índice general

Prólogo: Fundamentos del Aprendizaje por Refuerzo **1**

1. Probabilidad y Pensamiento Estocástico para RL **5**
1.1. ¿Por qué la programación tradicional no basta? . . . 5
1.2. Variables Aleatorias: Ponerle números al azar . . . 6
1.3. Probabilidad Condicional: Aprender con nueva información . . . 7
1.3.1. La Ley de Probabilidad Total . . . 7
1.4. El Concepto Más Importante: El Valor Esperado . . . 8
1.5. De la Teoría al Código: Ley de los Grandes Números . . . 9

2. El Problema de los Bandidos Multi-Brazo **11**
2.1. El Dilema Fundamental: Exploración frente a Explotación . . . 11
2.2. El Problema del Bandido Multi-Brazo (k-Armed Bandit) . . . 11
2.3. Retroalimentación Evaluativa e Instructiva . . . 13
2.4. Valor de Acción y Estimación del Rendimiento: $q_*(a)$ . . . 13
2.5. La Regla de Actualización Incremental . . . 14
2.6. Estrategia de Decisión: El Algoritmo ε-Greedy . . . 15
2.7. Implementación en Python: El Experimento de las Cuatro Tolvas . . . 16

3. Cadenas de Markov y Procesos Estocásticos **17**
3.1. De las Decisiones Aisladas a los Mundos en Movimiento . . . 17
3.2. La Propiedad de Markov: El Presente es lo Único que Importa . . . 18
3.3. La Matriz de Transición: El Mapa de Rutas en una Tabla . . . 19
3.4. Predicción Temporal: Viajando al Futuro Paso a Paso . . . 21
3.5. Distribución Estacionaria: Multiplicando la Matriz Hasta el Infinito . . . 23
3.5.1. El Gran Descubrimiento: Multiplicar la Matriz por Sí Misma (P^k) . . . 23
3.6. Implementación en Python: Proyección y Distribución Estacionaria . . . 25

4. Procesos de Decisión de Markov y Ecuaciones de Bellman **26**
4.1. La Unificación de Estados, Acciones y Recompensas . . . 26
4.1.1. La Hipótesis de la Recompensa . . . 26
4.2. Definición Formal de un MDP: La Tupla de Siete Elementos . . . 27
4.2.1. El Grafo del MDP: Estados, Acciones y Recompensas . . . 28
4.2.2. Representación Matricial del MDP . . . 28
4.2.3. Tabla de Parámetros del MDP . . . 29
4.2.4. El Retorno Descontado y la Preferencia Temporal . . . 29
4.3. Políticas y Funciones de Valor . . . 30
4.4. La Ecuación de Expectativa de Bellman . . . 30
4.5. Evaluación Iterativa de Políticas y Determinación de la Política Óptima . . . 30
4.5.1. Resolución Manual Paso a Paso: Aplicación en OptiBot . . . 31

4.5.2. Deducción de la Política Óptima 32
4.6. Implementación en Python: Iteración de Valor y Ecuaciones de Bellman 33

5. Aprendizaje por Refuerzo Tradicional: Algoritmo Q-Learning 34
5.1. De las Ecuaciones Analíticas al Aprendizaje sin Modelo (Model-Free) 34
5.2. El Laberinto de OptiBot: Un Almacén en Cuadrícula de 3×3 34
5.3. ¿Qué es exactamente la Tabla Q? La Memoria de Navegación 36
5.3.1. Estructura de la Tabla Q 36
5.3.2. La Regla de Decisión: Elegir la Flecha con Mayor Puntuación 36
5.4. La Ecuación de Q-Learning Explicada Paso a Paso 37
5.5. El Dilema: ¿Cómo Explorar el Laberinto? (Política ε-Greedy) 37
5.6. Cálculo Manual Paso a Paso: Siguiendo a OptiBot en el Laberinto 38
5.7. Evolución a Largo Plazo: Cómo los Ceros se Transforman en Valores Negativos . . 44
5.7.1. El auge de los pasillos interiores: Descubrimiento de dos rutas óptimas . . . 44
5.8. Visualización Gráfica: La Política Aprendida en la Cuadrícula 45
5.9. Implementación en Python: El Laberinto de OptiBot 46
5.10. La Gran Limitación: La Maldición de la Dimensionalidad 46

6. Aprendizaje por Refuerzo Profundo: Deep Q-Learning (DQN) 47
6.1. De Tablas Discretas a Aproximadores de Funciones Continuos 47
6.1.1. ¿Por qué las Tablas Q no Funcionan en el Mundo Real? 47
6.1.2. La Red Neuronal como Calculadora de Perillas Inteligentes 48
6.2. El Algoritmo Deep Q-Learning: Las Dos Redes y el Flujo Maestro 49
6.2.1. Fase 1: El Nacimiento de las Dos Redes Gemelas 51
6.2.2. Fase 2: Vivir la Aventura y Guardar Fotos en el Álbum 51
6.2.3. Fase 3: La Evaluación de Bellman y la Diana Fija 52
6.2.4. Fase 4: El Truco Maestro del Vector Target y el Gradiente 52
6.2.5. Fase 5: El Ciclo Continuo y la Sincronización Periódica 53
6.3. Cómo Aprende la Red: Función de Pérdida y Descenso de Gradiente 53
6.3.1. La Función de Pérdida: Midiendo la Sorpresa del Robot 53
6.3.2. ¿Por qué la sustitución aísla el error en la acción tomada? 54
6.3.3. El Enfoque del Descenso de Gradiente y el Semi-Gradiente 55
6.4. Cálculo Numérico Paso a Paso: Los 10 Pasos de DQN en OptiBot 56
6.5. Implementación Práctica: OptiBot DQN en el Cuaderno Interactivo 58
6.6. Comparativa Global: De las Tablas a las Redes Neuronales 59

1

Probabilidad y Pensamiento Estocástico para RL

1.1 ¿Por qué la programación tradicional no basta?

En la programación informática convencional, las cosas ocurren de manera predecible y ordenada. Si se escribe una instrucción para sumar dos cantidades o para aplicar un descuento del 15 %, el resultado será siempre exacto y reproducible. Si ese código se ejecuta diez veces o un millón de veces, la computadora devolverá con certeza absoluta la misma respuesta.

Sin embargo, cuando una máquina o un robot intenta tomar decisiones en el mundo real, esa certeza desaparece por completo:

- Una rueda puede patinar si el suelo tiene un poco de polvo, humedad o desgaste.
- Un sensor de distancia puede reportar dos metros y, un instante después, indicar dos metros y diez centímetros debido a un simple reflejo de luz.
- Un trayecto que usualmente toma tres minutos puede demorar diez si alguien dejó una caja bloqueando el pasillo.

Si se intentara programar una regla fija para cada uno de estos imprevistos, el código resultante sería infinito, frágil e inmanejable. Es aquí donde el Aprendizaje por Refuerzo recurre a la **probabilidad**: no como una colección de fórmulas abstractas para memorizar, sino como una brújula práctica que le permite a una máquina convivir con el azar, medir el riesgo y tomar decisiones inteligentes aunque no pueda predecir el futuro inmediato con exactitud.

A lo largo del curso se seguirá a **OptiBot**, un robot encargado de transportar paquetes desde el área de empaque hasta los muelles de carga en un almacén.
El entorno es cambiante: transitan personas, cruzan otros vehículos y surgen obstáculos imprevistos. Un programa con reglas fijas fallaría ante cualquier imprevisto. Para tomar decisiones acertadas, OptiBot necesita estimar probabilidades: saber qué eventos son comunes, cuáles representan riesgos y qué resultados esperar en promedio.

1.2 Variables Aleatorias: Ponerle números al azar

En el día a día ocurren eventos cuyo resultado final no se puede predecir con seguridad: el clima de mañana, el tiempo exacto que tardará un envío o la cara que mostrará una moneda al caer.

Para que una computadora pueda trabajar con esa incertidumbre, necesita transformar esos posibles resultados en valores numéricos. A esta herramienta se le denomina **variable aleatoria discreta** (representada convencionalmente con una letra mayúscula, como X):

> *Una variable aleatoria es simplemente una regla que asigna un número a cada resultado posible de una situación incierta.*

Por ejemplo, si se observa el tiempo de viaje que le toma a OptiBot llevar un paquete al muelle de carga, la variable aleatoria T podría tomar los valores discretos de 1, 2 o 3 (t) minutos.

A cada uno de esos valores posibles se le asocia su **probabilidad** de ocurrir, denotada formalmente como $P(X = x)$ o $P(T = t)$ en el caso del tiempo de viaje. Para que esta distribución tenga sentido lógico, debe cumplir con dos condiciones de sentido común:

1. **Las probabilidades se ubican siempre entre 0 y 1** (o entre 0 % y 100 %): No existen probabilidades negativas ni superiores al cien por ciento ($0 \leq P(X = x) \leq 1$). Si algo es imposible su probabilidad es 0; si es una certeza total, su probabilidad es 1.

2. **La suma de todos los casos posibles es igual a 1**: Si se toman en cuenta todos los resultados que pueden llegar a presentarse, es un hecho indiscutible que alguno de ellos terminará ocurriendo:

$$\sum_{x} P(X = x) = 1 \tag{1.1}$$

Ejemplo: La distribución de tiempos de viaje de OptiBot

Para ilustrar estos conceptos de forma concreta, supóngase que se cronometran 100 viajes de OptiBot transportando paquetes hacia el muelle de carga. Al clasificar los tiempos registrados, se obtiene el siguiente desglose:

Tiempo de viaje (t)	**Viajes observados**	**Probabilidad $P(T = t)$**
$t = 1$ minuto	20 de 100 viajes	$P(T = 1) = 0.20$ $(20\,\%)$
$t = 2$ minutos	50 de 100 viajes	$P(T = 2) = 0.50$ $(50\,\%)$
$t = 3$ minutos	30 de 100 viajes	$P(T = 3) = 0.30$ $(30\,\%)$

A partir de estos datos, se comprueban inmediatamente las dos propiedades obligatorias:

1. Cada probabilidad es un valor real entre 0 y 1 (0.20, 0.50 y 0.30).

2. La suma de todas las alternativas posibles totaliza la certeza absoluta (1.00 o $100\,\%$):

$$P(T = 1) + P(T = 2) + P(T = 3) = 0.20 + 0.50 + 0.30 = 1.00$$

Esta tabla describe la **distribución de probabilidad** de la variable T. Aunque no se puede adivinar con exactitud cuántos minutos tomará el próximo viaje en particular, el sistema ahora sabe que lo más habitual es tardar 2 minutos ($50\,\%$ de probabilidad), mientras que los viajes veloces de 1 minuto ocurrirán solo una de cada cinco veces ($20\,\%$).

1.3 Probabilidad Condicional: Aprender con nueva información

Las probabilidades rara vez permanecen fijas en el tiempo. Casi siempre se modifican en cuanto se obtiene nueva información del entorno.

Considérese un ejemplo cotidiano del almacén: ¿cuál es la probabilidad de que una rueda de OptiBot resbale? En condiciones normales es muy baja, tal vez de un $2\,\%$. Pero si los sensores detectan que el piso acaba de ser limpiado con jabón, esa probabilidad se incrementa de inmediato a un $40\,\%$. La probabilidad cambió radicalmente porque se obtuvo un dato clave sobre la situación actual del entorno.

Esta noción se expresa a través de la **probabilidad condicional**. La probabilidad de que ocurra un evento B, dado que ya ocurrió o se conoce la ocurrencia previa del evento A, se escribe $P(B \mid A)$ y se calcula mediante:

$$P(B \mid A) = \frac{P(A \cap B)}{P(A)} \tag{1.2}$$

Donde $P(A \cap B)$ representa la probabilidad de que ambos eventos ocurran de manera conjunta. En el Aprendizaje por Refuerzo, esta fórmula es la piedra angular de todo el razonamiento: el agente se pregunta en cada paso: *«¿Cuál es la probabilidad de alcanzar una meta exitosa (B), dado que se decidió ejecutar la acción de girar a la izquierda (A)?»*.

Ejemplo: Cálculo numérico de probabilidad condicional

Para observar cómo opera esta fórmula en la práctica, considérese un registro histórico de 100 despachos realizados por OptiBot:

- Se define el evento A: *«El robot toma la ruta del pasillo central»*. Esto ocurrió en 40 de los 100 viajes, por lo que $P(A) = \frac{40}{100} = 0.40$ $(40\,\%)$.
- Se define el evento B: *«El viaje sufre una demora considerable»*.
- Se constató que en 16 de esos 100 viajes el robot tomó el pasillo central **y además** sufrió la demora. Es decir, la probabilidad conjunta es $P(A \cap B) = \frac{16}{100} = 0.16$ $(16\,\%)$.

Si OptiBot decide ingresar al pasillo central, ¿cuál es la probabilidad de que termine demorándose? Aplicando la definición de probabilidad condicional:

$$P(B \mid A) = \frac{P(A \cap B)}{P(A)} = \frac{0.16}{0.40} = 0.40 \quad (40\,\%)$$

Interpretación práctica: En todo el almacén, la probabilidad general de sufrir demoras tal vez sea de apenas un $24\,\%$. No obstante, en cuanto se confirma que el robot eligió el pasillo central (evento A), el riesgo de demora se eleva automáticamente al $40\,\%$. Esta actualización de probabilidades es la que le permite al agente aprender qué acciones son seguras y cuáles aumentan el riesgo de penalización.

1.3.1 La Ley de Probabilidad Total

En muchas ocasiones se desea determinar la probabilidad general de que ocurra un suceso, sabiendo que este puede ocurrir a través de distintas rutas o circunstancias intermedias.

Si OptiBot puede desplazarse hacia el muelle de carga eligiendo entre tres rutas alternativas como el pasillo central, el pasillo norte o el pasillo sur; la probabilidad total de que ocurra un evento

(como sufrir una demora) se obtiene evaluando cada alternativa por separado y sumando sus contribuciones ponderadas:

$$P(B) = \sum_{i=1}^{n} P(B \mid A_i) \cdot P(A_i) \tag{1.3}$$

En términos prácticos: se calcula la probabilidad de que ocurra el evento en cada camino posible, se multiplica por la frecuencia con la que se transita por dicho camino, y se suman todas esas partes.

Ejemplo: Probabilidad total de sufrir demoras en el almacén

Siguiendo con el caso de OptiBot, supóngase que el robot dispone de tres rutas posibles hacia el muelle de carga (A_1, A_2 y A_3):

- **Pasillo Central (A_1):** Es la ruta directa. OptiBot la utiliza el $40\,\%$ de las veces ($P(A_1) = 0.40$). Como se analizó en el ejemplo anterior, al ser muy concurrida presenta una probabilidad de demora del $40\,\%$ ($P(B \mid A_1) = 0.40$).
- **Pasillo Norte (A_2):** Es una ruta más amplia y despejada. El robot la utiliza también el $40\,\%$ de las veces ($P(A_2) = 0.40$). En este pasillo la probabilidad de demora es baja, de solo un $10\,\%$ ($P(B \mid A_2) = 0.10$).
- **Pasillo Sur (A_3):** Es una ruta secundaria que se emplea el $20\,\%$ restante de las veces ($P(A_3) = 0.20$). Su probabilidad de demora es intermedia, del $20\,\%$ ($P(B \mid A_3) = 0.20$).

Para hallar la probabilidad total de que un viaje sufra demoras, sin importar qué pasillo haya elegido el robot, se aplica la Ley de Probabilidad Total:

$$\begin{aligned} P(B) &= P(B \mid A_1) \cdot P(A_1) + P(B \mid A_2) \cdot P(A_2) + P(B \mid A_3) \cdot P(A_3) \\ &= (0.40 \times 0.40) + (0.10 \times 0.40) + (0.20 \times 0.20) \\ &= 0.16 + 0.04 + 0.04 \\ &= \mathbf{0.24} \quad (\mathbf{24}\,\%) \end{aligned}$$

Conclusión didáctica: En promedio, el $24\,\%$ de todos los viajes en el almacén sufrirán demoras. Esta visión global le permite al algoritmo comprender el rendimiento general de su entorno antes de comenzar a optimizar qué porcentaje de viajes le conviene destinar a cada pasillo.

> Esta suma ponderada no es una curiosidad abstracta. Representa el núcleo matemático sobre el cual se construyen los algoritmos de decisión en Inteligencia Artificial. En el Capítulo 4, se comprobará que la célebre **Ecuación de Bellman** aplica exactamente esta misma lógica para calcular el valor de cualquier estado futuro.

1.4 El Concepto Más Importante: El Valor Esperado

Si una persona compra un boleto de lotería y gana un premio pequeño, cometería una equivocación grave si concluyera que comprar lotería todos los días es una inversión financiera rentable. Juzgar una estrategia basándose únicamente en el resultado de un intento aislado suele conducir a decisiones erróneas.

En el Aprendizaje por Refuerzo, a un robot no le interesa ganar una recompensa por pura casualidad en una sola ocasión; lo que busca es encontrar una estrategia que produzca los mejores resultados posibles **en promedio a largo plazo**. Para medir este rendimiento sostenido se utiliza el **valor esperado** (denotado matemáticamente como $\mathbb{E}[X]$):

$$\mathbb{E}[X] \doteq \sum_{x} x \cdot P(X = x) \tag{1.4}$$

El símbolo $\doteq$ (un signo de igual con un punto encima) es una convención muy común en la literatura de Aprendizaje por Refuerzo y significa simplemente **«se define como»**. Indica que la fórmula no es el resultado de un despeje matemático previo, sino la propia definición fundamental del concepto.

El valor esperado no pretende adivinar qué valor exacto se observará en el siguiente viaje; lo que indica con precisión es el promedio al que tenderán los resultados si la acción se repite cientos o miles de veces bajo condiciones similares.

Ejemplo: Cálculo del valor esperado en los envíos de OptiBot

Para comprender cómo se aplica esta fórmula en la toma de decisiones, considérese el balance económico diario de OptiBot al transportar pedidos hacia los camiones de reparto. Se presentan tres escenarios posibles en cada viaje:

- **Entrega rápida (10 minutos):** el cliente abona una tarifa preferencial y deja una ganancia neta de $+\$10$. Esto ocurre el $26\,\%$ de las veces ($P = 0.26$).
- **Entrega regular (20 minutos):** el viaje se completa dentro del horario estándar y deja una ganancia de $+\$5$. Esto ocurre el $50\,\%$ de las veces ($P = 0.50$).
- **Entrega con demora (más de 30 minutos):** el almacén debe compensar al cliente con un cupón, resultando en una pérdida de $-\$2$. Esto ocurre el $24\,\%$ de las veces ($P = 0.24$).

Para calcular cuánta ganancia genera OptiBot en promedio por cada paquete transportado, se multiplica cada resultado económico por su respectiva probabilidad y se suman:

$$\begin{aligned}\mathbb{E}[\text{Ganancia}] &= (10 \times 0.26) + (5 \times 0.50) + (-2 \times 0.24) \\ &= 2.60 + 2.50 - 0.48 \\ &= \$\mathbf{4.62} \text{ por paquete}\end{aligned}$$

Conclusión didáctica: Obsérvese un detalle fundamental: **ningún paquete individual produce exactamente** $\$4.62$. Cada envío aislado deja una ganancia de $\$10$, de $\$5$ o una pérdida de $-\$2$. Sin embargo, si OptiBot transporta 1,000 paquetes a lo largo del mes, la empresa habrá acumulado aproximadamente $\$4,620$. Esta cifra promedio es la que le permite al algoritmo decidir objetivamente si una ruta o una política de trabajo es beneficiosa.

1.5 De la Teoría al Código: Ley de los Grandes Números

Una duda natural es: ¿realmente se cumple este promedio teórico cuando el sistema opera en la práctica?

La **Ley de los Grandes Números** responde afirmativamente: a medida que un experimento se repite una gran cantidad de veces, la media observada en la realidad se aproxima de manera casi perfecta al valor esperado teórico calculado en el papel.

Para experimentar de forma interactiva con esta simulación, el código completo se encuentra disponible en el cuaderno Jupyter:

`notebooks/cap00_probabilidad.ipynb`

En este cuaderno interactivo se simula el trabajo de OptiBot realizando $50{,}000$ despachos en el almacén y se grafica la convergencia del promedio acumulado en tiempo real. Al ejecutar la simulación, se comprueba que el promedio observado difiere del valor teórico por menos de dos centavos de dólar ($\approx \$4.62$), validando empíricamente la teoría.

Con esta base conceptual, el azar deja de ser un obstáculo desconcertante y se transforma en una variable cuantificable y predecible a gran escala. En el Capítulo 2, estas herramientas permitirán analizar el primer problema clásico del aprendizaje por refuerzo: cómo debe actuar un agente inteligente ante el dilema entre **explotar** una opción que ya sabe que funciona o **explorar** nuevas alternativas para descubrir si ofrecen ganancias superiores.

2

El Problema de los Bandidos Multi-Brazo

2.1 El Dilema Fundamental: Exploración frente a Explotación

La toma de decisiones en entornos inciertos plantea una disyuntiva constante: el conflicto entre la **exploración** y la **explotación**. Este problema se manifiesta siempre que un agente (como un robot o un algoritmo) debe elegir entre dos alternativas opuestas:

1. **Explotación:** Elegir la opción que ha demostrado el mejor rendimiento hasta el momento, con el fin de asegurar una ganancia o beneficio ya conocido.

2. **Exploración:** Probar alternativas distintas cuyo rendimiento todavía no se conoce con certeza, asumiendo el riesgo de obtener un mal resultado a cambio de la oportunidad de descubrir una opción superior.

Para evaluar el resultado de cada elección, el entorno devuelve una señal numérica de beneficio o costo, denotada convencionalmente como R_t (del término en inglés *reward* o retorno): un valor positivo representa una ganancia o premio, mientras que un valor negativo refleja una penalización o costo.

Para visualizar este dilema de forma intuitiva, considérese la situación ilustrada en la Figura 2.1, donde un agente se encuentra frente a una bifurcación en un laberinto. Si el agente se limita a explotar su conocimiento previo, repetirá una y otra vez el camino hacia el cofre, perdiéndose para siempre la posibilidad de encontrar el diamante. Por el contrario, si se dedica exclusivamente a explorar senderos desconocidos, correrá el riesgo constante de elegir rutas vacías o peligrosas sin llegar a consolidar sus ganancias. El verdadero desafío del Aprendizaje por Refuerzo radica en balancear inteligentemente ambas actitudes.

2.2 El Problema del Bandido Multi-Brazo (k-Armed Bandit)

Para comprender el conflicto entre explorar y explotar sin enredarse con mapas o laberintos complejos, el Aprendizaje por Refuerzo parte del modelo más simple que existe: el **Bandido Multi-Brazo**.

Este nombre tan particular surge de una comparación clásica con los casinos. En inglés, a las máquinas tragamonedas tradicionales se las llama comúnmente «bandidos de un brazo» (*one-armed bandits*), ya que funcionan con una palanca lateral (un «brazo») y suelen quedarse con las monedas de los jugadores. Cuando se tienen varias máquinas alineadas (o una sola máquina con varias palancas), se forma un «bandido de múltiples brazos».



Figura 2.1: Dilema entre exploración y explotación en un laberinto. Al optar por la **explotación** (camino inferior), el agente sigue las huellas conocidas hacia un cofre seguro con una ganancia ya comprobada. En cambio, si decide **explorar** (camino superior), asume la incertidumbre de adentrarse en un sendero nuevo que desconoce, lo que le brinda la oportunidad de descubrir un beneficio potencialmente superior (el diamante brillante).

Ejemplo: Las máquinas tragamonedas del casino

Para entender cómo funciona este escenario, considérese a una persona frente a una fila de k máquinas tragamonedas:

- Cada máquina entrega premios al azar, pero no todas pagan igual: algunas son generosas con frecuencia, mientras que otras casi nunca dan nada. El jugador no sabe de antemano cuál es la más conveniente.
- El jugador cuenta con una cantidad fija de monedas y, en cada intento, solo puede accionar una sola palanca.
- Supóngase que en su primer intento jala la palanca de la máquina #1 y gana $\$5$.

En ese mismo instante aparece el dilema:

- **Si opta por la explotación:** Puede seguir jugando siempre en la máquina #1 para asegurarse esa ganancia conocida de $\$5$ por jugada. Sin embargo, nunca descubrirá si la máquina #3 entrega en promedio premios mucho mayores, como $\$20$.
- **Si opta por la exploración:** Probará las demás máquinas para averiguar cuánto pagan realmente, asumiendo el riesgo de perder monedas en opciones malas que apenas entreguen beneficios.

2.3 Retroalimentación Evaluativa e Instructiva

Una distinción conceptual esencial entre el aprendizaje supervisado convencional y el aprendizaje por refuerzo reside en el tipo de retroalimentación que recibe el sistema:

- En el **aprendizaje supervisado**, el sistema recibe una señal **instructiva**: existe una respuesta correcta predeterminada (una etiqueta) que indica explícitamente cuál debió haber sido la salida ideal.
- En el **aprendizaje por refuerzo**, no hay un tutor que indique la respuesta óptima. El agente recibe una señal puramente **evaluativa**: el entorno devuelve un valor numérico R_t que califica el desempeño de la acción tomada, pero no le dice al agente si alguna de las otras opciones disponibles habría producido un mejor resultado.

El agente debe, por tanto, descubrir de manera autónoma cuáles decisiones resultan más rentables mediante la interacción directa y el método de prueba y error.

En el centro logístico, OptiBot opera en la estación de despacho. Ante cada paquete que finaliza su proceso de embalaje, el robot debe seleccionar una de **4 empresas de mensajería** disponibles:

- **Proveedor A:** Costos moderados y tiempos estables.
- **Proveedor B:** Servicio eficiente con la mayor rentabilidad esperada promedio.
- **Proveedor C:** Tarifas reducidas pero con alta tasa de demoras y penalizaciones.
- **Proveedor D:** Servicio premium constante con costos elevados.

OptiBot **no dispone a priori** de los rendimientos esperados de cada empresa. Cada despacho genera una ganancia o pérdida neta R_t sujeta a fluctuaciones estocásticas. El objetivo del robot consiste en maximizar el beneficio acumulado a lo largo de un horizonte de 2,000 despachos.

2.4 Valor de Acción y Estimación del Rendimiento: $q_*(a)$

Para tomar buenas decisiones, el agente necesita evaluar qué tan conveniente es cada opción. En cada paso de tiempo t (cada turno o despacho), intervienen dos variables fundamentales:

- A_t: la **acción** seleccionada por el agente en el instante t (por ejemplo, elegir una empresa de mensajería o tirar de una palanca).
- R_t: la **recompensa** numérica recibida inmediatamente después como consecuencia de esa acción (puede ser una ganancia o un costo).

1. El valor real u objetivo: $q_*(a)$

Cada acción posible a dentro del conjunto de opciones disponibles $\mathcal{A}$ tiene un **valor verdadero**, denotado como $q_*(a)$. Este valor representa la recompensa promedio teórica que obtendríamos a largo plazo si eligiéramos esa misma acción infinitas veces. Matemáticamente se define como:

$$q_*(a) \doteq \mathbb{E}\left[R_t \mid A_t = a\right] \tag{2.1}$$

¿Cómo se lee y qué significa cada símbolo de esta ecuación?

- $q_*(a)$ (*«q-estrella de a»*): Es el **valor real y objetivo** de la acción a. El asterisco $*$ indica que es el valor perfecto o de referencia (la verdad oculta en el entorno).
- $\doteq$ (*«se define como»*): Este signo de igual con un punto encima indica una definición formal.
- $\mathbb{E}[\cdot]$ (*«esperanza matemática»*): Es el **promedio esperado** o valor medio a largo plazo.
- $R_t \mid A_t = a$ (*«R_t dado que A_t es igual a a»*): La barra vertical $\mid$ se lee **«dado que»** o «condicionado a». La expresión completa responde a: *«Si elegimos la acción a, ¿cuál es la recompensa promedio que esperamos recibir?»*.

2. La estimación aprendida por el agente: $Q_t(a)$

En la práctica, el agente no conoce $q_*(a)$ de antemano. Por ello, mantiene en memoria una **estimación empírica** llamada $Q_t(a)$, construida a partir de las recompensas que ha recibido en el pasado.
En palabras simples:

$$Q_t(a) = \frac{\text{Suma de recompensas obtenidas al elegir la acción } a}{\text{Número de veces que se ha elegido la acción } a}$$

De manera matemática formal, si la acción a se ha seleccionado $N_t(a)$ veces antes del instante t, su estimación se calcula como:

$$Q_t(a) \doteq \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbb{I}(A_i = a)}{N_t(a)} \tag{2.2}$$

¿Cómo se lee y qué representa cada parte de esta fórmula?

- $Q_t(a)$ (*«Q en el tiempo t de la acción a»*): Es la **estimación o creencia actual** que tiene el agente en su memoria sobre la calidad de la acción a, justo antes de jugar el turno t.
- $N_t(a)$ (*«N en el tiempo t de a»*): Es un simple **contador** que indica cuántas veces se ha ejecutado la acción a antes del paso t.
- $\sum_{i=1}^{t-1}$ (*«sumatoria desde el paso $i = 1$ hasta $t - 1$»*): Indica que se suman los resultados de todos los turnos previos al turno actual t.
- $\mathbb{I}(A_i = a)$ (*«función indicadora»*): Actúa como un **filtro o interruptor**: vale 1 si en el paso i se eligió la acción a, y vale 0 si se eligió otra. De este modo, la suma solo acumula las recompensas que realmente corresponden a la acción a.

2.5 La Regla de Actualización Incremental

Almacenar la secuencia histórica completa de recompensas para cada acción demandaría un consumo de memoria linealmente creciente con el tiempo. Resulta más eficiente computacionalmente derivar una formulación recursiva que actualice la media empleando únicamente la

estimación anterior y la nueva observación:

$$\begin{aligned} Q_{n+1} &= \frac{1}{n}\sum_{i=1}^{n} R_i \\ &= \frac{1}{n}\left(R_n + \sum_{i=1}^{n-1} R_i\right) \\ &= \frac{1}{n}\left(R_n + (n-1)Q_n\right) \\ &= Q_n + \frac{1}{n}\left(R_n - Q_n\right) \end{aligned}$$

La ecuación resultante establece el patrón de actualización paradigmático de todo el Aprendizaje por Refuerzo:

$$\text{NuevaEstimación} \leftarrow \text{EstimaciónPrevia} + \alpha \cdot \Big[\text{Objetivo} - \text{EstimaciónPrevia}\Big] \tag{2.3}$$

La diferencia [Objetivo − EstimaciónPrevia] constituye el **error de estimación** (o error de predicción), el cual cuantifica la sorpresa generada por el entorno. El parámetro α determina la tasa de aprendizaje o tamaño de paso.

En entornos no estacionarios (donde las recompensas de cada acción pueden cambiar con el tiempo), usar $\frac{1}{n}$ haría que el agente se vuelva rígido y deje de aprender a medida que n crece. Por ello, se reemplaza $\frac{1}{n}$ por un tamaño de paso constante $\alpha \in (0, 1]$. De este modo, mientras más grande sea α, mayor prioridad le dará a las experiencias recientes y «olvidará» más rápido el pasado lejano (*decaimiento exponencial*), manteniéndose flexible ante cualquier cambio.

2.6 Estrategia de Decisión: El Algoritmo ε-Greedy

Si el agente siempre elige la acción que parece mejor en ese momento (*estrategia codiciosa o greedy*, $\arg\max_a Q(a)$), corre el riesgo de quedarse atrapado en una opción mediocre solo porque las demás tuvieron mala suerte en sus primeros intentos.

El algoritmo ε**-greedy** resuelve esto con una regla muy sencilla: la mayor parte del tiempo elige la mejor opción conocida (**explotación**), pero deja una pequeña probabilidad ε (por ejemplo, el $10\,\%$ de las veces) para probar una acción al azar (**exploración**):

$$A_t = \begin{cases} \text{acción aleatoria (explorar)}, & \text{con probabilidad } \varepsilon, \\ \arg\max_{a \in \mathcal{A}} Q_t(a) \text{ (explotar la mejor)}, & \text{con probabilidad } 1-\varepsilon. \end{cases} \tag{2.4}$$

Veamos cómo actualiza OptiBot sus estimaciones en sus tres primeros despachos, partiendo de estimaciones en cero $Q = [0.0, 0.0, 0.0, 0.0]$ para los Proveedores A, B, C y D:

1. **Paso 1 (Exploración):** Elige al azar al **Proveedor B** ($a = 2$) y obtiene una recompensa $R_1 = +5.20$. Al ser su primer uso ($N(B) = 1$):

$$Q_2(B) = 0.0 + \frac{1}{1}(5.20 - 0.0) = +\mathbf{5.20}$$

2. **Paso 2 (Explotación):** Como B tiene la estimación más alta (5.20), lo explota. Obtiene $R_2 = +4.40$. Al ser su segundo uso ($N(B) = 2$):

$$Q_3(B) = 5.20 + \frac{1}{2}(4.40 - 5.20) = 5.20 - 0.40 = +\mathbf{4.80}$$

3. **Paso 3 (Exploración):** Decide explorar y prueba al **Proveedor A** ($a = 1$). Obtiene $R_3 = +3.00$. Al ser su primer uso ($N(A) = 1$):

$$Q_4(A) = 0.0 + \frac{1}{1}(3.00 - 0.0) = +\mathbf{3.00}$$

Resultado: La memoria del robot queda en $Q = [3.00,\ 4.80,\ 0.00,\ 0.00]$. OptiBot seguirá explotando preferentemente al Proveedor B (por tener el valor más alto, 4.80), pero el mecanismo ε le permitirá explorar oportunamente a C y D.

2.7 Implementación en Python: El Experimento de las Cuatro Tolvas

Para experimentar de forma interactiva con el algoritmo de bandidos multi-brazo y la regla ε-greedy, el código completo se encuentra disponible en el cuaderno Jupyter:

`notebooks/cap01_bandidos.ipynb`

En este cuaderno se implementa la clase `BandidoAlmacen`, se simulan $2{,}000$ despachos de paquetes en el almacén y se visualizan gráficas comparativas entre los valores reales subyacentes $q^*(a)$ y las estimaciones aprendidas $Q(a)$, así como la frecuencia con la que OptiBot explota la mejor tolva frente a las exploraciones aleatorias.

El modelo de bandidos multi-brazo permite formalizar con rigor el dilema entre exploración y explotación. No obstante, presenta una limitación estructural clave: **las acciones del agente no alteran el estado del entorno**.
En el mundo real, los entornos no son estáticos: evolucionan y cambian de estado con el tiempo (por ejemplo, una máquina que se desgasta o la batería que se agota). Antes de aprender a tomar decisiones en ellos, en el Capítulo 3 se estudiará cómo modelar estos cambios mediante las **Cadenas de Markov**.

3

Cadenas de Markov y Procesos Estocásticos

3.1 De las Decisiones Aisladas a los Mundos en Movimiento

En el Capítulo 2 vimos cómo un agente aprende a elegir la mejor opción mediante ensayo y error (el problema de los bandidos). Sin embargo, en ese escenario asumíamos una simplificación muy grande: cada intento comenzaba desde cero, como si lanzáramos una moneda al aire una y otra vez. Elegir una máquina tragamonedas o un proveedor de envíos no cambiaba la hora del día, no desgastaba los motores del robot ni modificaba el lugar donde estaba parado.

En el mundo real, los entornos tienen **memoria física, inercia y desgaste**:

- Si un robot tiene la batería al $95\,\%$, puede realizar rutas largas con tranquilidad; pero si su batería cae al $5\,\%$, su prioridad urgente debe ser buscar un cargador.
- Si el suelo del almacén se moja con aceite, los neumáticos resbalarán con mayor facilidad en los siguientes giros.
- Una máquina industrial no se rompe por sorpresa de un segundo a otro: primero se calienta, luego vibra, sus rodamientos se desgastan y, finalmente, se descompone si nadie interviene.

A cada una de estas situaciones o condiciones en las que puede encontrarse un sistema en un instante determinado se le llama **estado**. Antes de enseñarle a un robot a tomar decisiones inteligentes para cambiar su entorno, necesitamos entender las reglas matemáticas que describen cómo evoluciona ese entorno por sí mismo a lo largo del tiempo. A este modelo se le conoce como una **Cadena de Markov**.

En el almacén de distribución donde opera nuestro robot **OptiBot**, la pieza central de la logística es una gran cinta transportadora encargada de mover cajas pesadas. Los rodillos mecánicos de esta cinta giran sin parar durante turnos enteros. Con el paso de las horas, los rodamientos de los rodillos sufren desgaste continuo.
OptiBot necesita aprender a predecir cuándo fallará la maquinaria para evitar que la operación se detenga. ¿Cómo podemos modelar el envejecimiento de esa máquina paso a paso?

3.2 La Propiedad de Markov: El Presente es lo Único que Importa

Imagine que va conduciendo un automóvil por la autopista a 80 km/h y desea predecir en qué punto de la carretera estará dentro de exactamente un segundo. ¿Necesita revisar en su memoria qué desayunó por la mañana o por cuántas ciudades pasó hace tres días? Por supuesto que no. Para saber dónde estará en el siguiente segundo, solo necesita conocer dos datos del **instante presente**: su posición y su velocidad actuales.

Esta intuición tan natural es el corazón de la **Propiedad de Markov**:

> *«El futuro depende únicamente del estado presente, y no de toda la historia pasada que condujo a él.»*

De manera matemática formal, si llamamos S_t al estado actual en el tiempo t, la probabilidad de llegar al estado futuro S_{t+1} se escribe:

$$P\left(S_{t+1} = s_{t+1} \mid S_t = s_t,\, S_{t-1} = s_{t-1},\, \ldots,\, S_0 = s_0\right) = P\left(S_{t+1} = s_{t+1} \mid S_t = s_t\right) \tag{3.1}$$

¿Cómo se lee y qué significa cada símbolo de esta ecuación?

- $S_{t+1} = s_{t+1}$ (*«el estado en el turno $t+1$»*): Es el **futuro inmediato** (¿dónde estará el sistema en el siguiente paso?).
- $S_t = s_t$ (*«el estado en el turno t»*): Es el **presente** (¿dónde está el sistema justo ahora?).
- $S_{t-1}, \ldots, S_0$ (*«los estados previos hasta el inicio»*): Representa toda la **historia pasada** del sistema.
- La barra vertical $\mid$ (*«dado que»*): Indica que ya conocemos esa información previa.
- El signo de igualdad ($=$): Es la clave de todo. Afirma que conocer el historial pasado completo **no agrega ninguna información nueva ni cambia las probabilidades** si ya conocemos con exactitud el estado presente.

Un sistema markoviano no necesita guardar un registro infinito de todo lo que ocurrió en el pasado. Toda la información acumulada que resulta relevante para predecir el futuro ya se encuentra resumida y concentrada dentro del **estado presente**. En términos técnicos se dice que el estado presente es una *estadística suficiente* del proceso.

Ejemplo: El sistema de rodamientos en la cinta transportadora de OptiBot

Para aterrizar esta idea en un caso práctico, observemos los rodamientos de la cinta transportadora de OptiBot. Al final de cada turno de trabajo, el estado de salud de un rodamiento se clasifica en una de tres categorías dentro del conjunto de estados:

$$\mathcal{S} = \{\text{Nuevo } (N),\ \text{Desgastado } (D),\ \text{Averiado } (A)\}$$

1. **Estado Nuevo (N):** La pieza está recién instalada y funciona con suavidad.
   - Tiene un $70\,\%$ de probabilidad de mantenerse en buen estado al terminar el turno ($N \to N$).
   - Tiene un $20\,\%$ de probabilidad de comenzar a mostrar fatiga y pasar a desgastado ($N \to D$).

- Tiene un $10\,\%$ de probabilidad de sufrir una fractura súbita y romperse directamente ($N \to A$).

2. **Estado Desgastado (D):** La pieza rechina y vibra, pero aún permite mover la cinta.
   - Tiene un $60\,\%$ de probabilidad de seguir desgastada un turno más ($D \to D$).
   - Tiene un $30\,\%$ de probabilidad de fallar por completo y romperse ($D \to A$).
   - Tiene un $10\,\%$ de probabilidad de que un técnico aplique lubricante y la devuelva al estado nuevo ($D \to N$).
3. **Estado Averiado (A):** El rodamiento se trabó y la cinta transportadora está paralizada.
   - Tiene un $90\,\%$ de probabilidad de continuar averiado debido a demoras en la llegada del repuesto ($A \to A$).
   - Tiene un $10\,\%$ de probabilidad de ser reemplazado por un repuesto nuevo durante el turno ($A \to N$).
   - No transita directamente a desgastado ($0\,\%$), pues una pieza rota solo vuelve a operar cuando se sustituye por una totalmente nueva.

Este comportamiento estocástico se visualiza de forma intuitiva como un mapa de carreteras en el **grafo de transición de estados** de la Figura 3.1. Cada círculo representa un estado y cada flecha indica la probabilidad de saltar de un estado a otro en el siguiente turno.

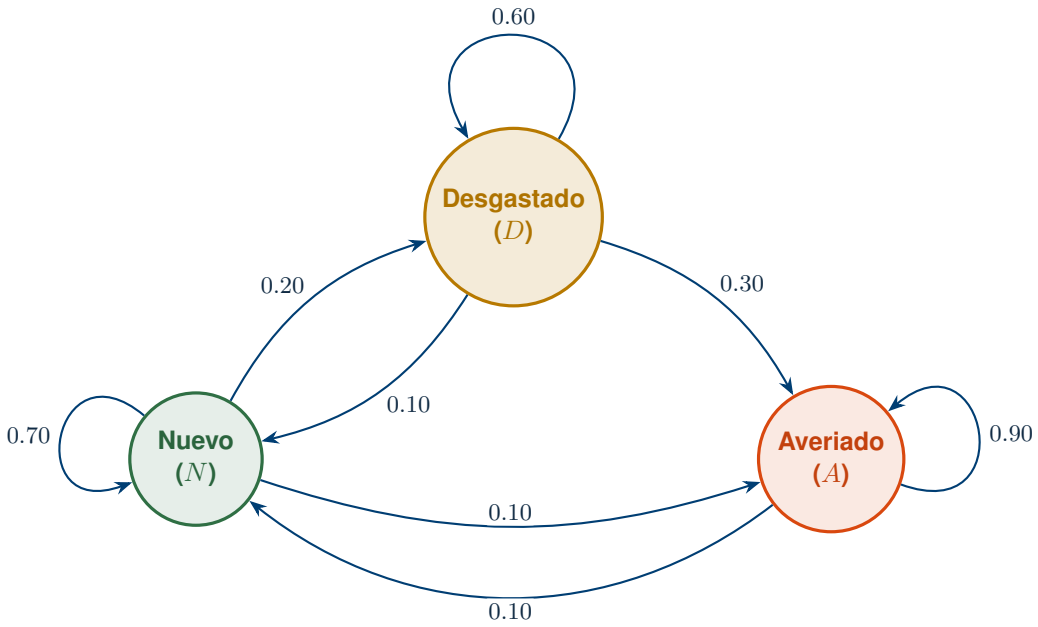


Figura 3.1: Grafo de transición de estados para el rodamiento de la cinta transportadora en el almacén de OptiBot. Las flechas representan los saltos posibles entre turnos y los números indican sus probabilidades (P_{ij}).

3.3 La Matriz de Transición: El Mapa de Rutas en una Tabla

Dibujar un grafo es excelente para entender la idea general, pero para hacer cálculos rápidos en una computadora resulta mucho más práctico organizar todas las probabilidades en una tabla numérica: la **Matriz de Transición de Estados** P.

Para un sistema con n estados posibles $\mathcal{S} = \{s_1, s_2, \ldots, s_n\}$, cada celda P_{ij} (fila i, columna j) se define como:

$$P_{ij} \doteq P(S_{t+1} = s_j \mid S_t = s_i) \tag{3.2}$$

Reuniendo todas las probabilidades condicionales en una sola estructura, obtenemos la matriz general de transición P de tamaño $n \times n$:

$$P = \begin{pmatrix} P_{11} & P_{12} & \ldots & P_{1n} \\ P_{21} & P_{22} & \ldots & P_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ P_{n1} & P_{n2} & \ldots & P_{nn} \end{pmatrix} \tag{3.3}$$

¿Cómo entender y leer esta matriz sin confundirse jamás?

- **La Fila i representa el Origen:** ¿En qué estado se encuentra la máquina **hoy** ($S_t = s_i$)?
- **La Columna j representa el Destino:** ¿A qué estado pasará la máquina **mañana** ($S_{t+1} = s_j$)?
- **Regla mnemotécnica de oro:** *«Fila = ¿Dónde estoy? → Columna = ¿A dónde voy?»*. Cada celda P_{ij} responde directamente a: *«Estando en el estado i, ¿cuál es la probabilidad de brincar al estado j?»*.

Propiedad Obligatoria: Cada Fila Debe Sumar Exactamente 1.0 (100 %)

Piénselo por un momento: si la máquina está hoy en el estado i, al siguiente turno obligatoriamente tiene que terminar en alguno de los n estados posibles (la máquina no puede evaporarse ni desaparecer en la nada). Por lo tanto, si sumamos todas las probabilidades al salir de cualquier fila i, la suma total debe ser siempre igual a 1.0:

$$\sum_{j=1}^{n} P_{ij} = 1 \quad \text{para toda fila } i \tag{3.4}$$

A cualquier matriz con números positivos o cero donde cada fila sume exactamente 1.0 se le llama en matemáticas una **matriz estocástica** (o matriz de probabilidades).

Ejemplo: Construcción de la matriz de transición de OptiBot

Ordenando los tres estados como $(\text{Nuevo}, \text{Desgastado}, \text{Averiado})$, la matriz P queda organizada de la siguiente forma:

$$P = \begin{pmatrix} 0.70 & 0.20 & 0.10 \\ 0.10 & 0.60 & 0.30 \\ 0.10 & 0.00 & 0.90 \end{pmatrix} \begin{array}{l} \leftarrow \text{Fila 1 } (N)\text{: } 0.70 + 0.20 + 0.10 = \mathbf{1.0} \\ \leftarrow \text{Fila 2 } (D)\text{: } 0.10 + 0.60 + 0.30 = \mathbf{1.0} \\ \leftarrow \text{Fila 3 } (A)\text{: } 0.10 + 0.00 + 0.90 = \mathbf{1.0} \end{array} \tag{3.5}$$

Ejemplos de lectura directa en la matriz:

- $P_{12} = 0.20$ (Fila 1, Columna 2): Estando en **Nuevo**, hay un $20\,\%$ de probabilidad de pasar a **Desgastado**.
- $P_{23} = 0.30$ (Fila 2, Columna 3): Estando en **Desgastado**, hay un $30\,\%$ de probabilidad de terminar **Averiado**.
- $P_{32} = 0.00$ (Fila 3, Columna 2): Estando en **Averiado**, hay un $0\,\%$ de probabilidad de pasar a desgastado sin antes haber sido reparado a nuevo.

3.4 Predicción Temporal: Viajando al Futuro Paso a Paso

Una vez que tenemos la matriz de transición P, podemos predecir con exactitud matemática el futuro del sistema a cualquier número de turnos.

El Vector de Estado Inicial (μ_0)

Para saber a dónde iremos, primero debemos saber de dónde partimos en el instante inicial ($t = 0$). Esto se escribe en un vector fila de probabilidades llamado μ_0 (*«mu sub-cero»*):

$$\mu_0 = \big[P(S_0 = N),\ P(S_0 = D),\ P(S_0 = A)\big] \tag{3.6}$$

- **Certeza de pieza nueva:** Si acabamos de instalar un rodamiento nuevo de paquete, sabemos al $100\,\%$ que está en N:
$$\mu_0 = [1.0,\ 0.0,\ 0.0]$$
- **Incertidumbre de inicio:** Si compramos un lote usado donde hay $50\,\%$ de probabilidad de que esté nuevo y $50\,\%$ de que esté desgastado, escribiríamos:
$$\mu_0 = [0.50,\ 0.50,\ 0.00]$$

La Regla de Proyección: Ecuaciones de Chapman-Kolmogorov

¿Cómo calculamos la distribución de probabilidades para el siguiente turno ($t = 1$)? Basta con multiplicar el vector inicial por la matriz de transición:

$$\mu_1 = \mu_0 P \tag{3.7}$$

¿Y para el segundo turno ($t = 2$)? Multiplicamos el resultado anterior nuevamente por la matriz:

$$\mu_2 = \mu_1 P = (\mu_0 P)P = \mu_0 P^2 \tag{3.8}$$

Siguiendo esta regla para cualquier turno futuro k (llamada en la literatura matemática las **ecuaciones de Chapman-Kolmogorov**):

$$\mu_{\mathbf{k}} = \mu_{\mathbf{0}}\mathbf{P}^{\mathbf{k}} \tag{3.9}$$

¡Esta fórmula es sumamente poderosa! Significa que para conocer el estado de la máquina dentro de 100 turnos, no tenemos que simular paso a paso cada minuto; basta con elevar la matriz a la potencia 100 (P^{100}) y multiplicarla por el estado de partida.

Ejemplo: Cálculo manual de la evolución a corto plazo ($t = 1$ y $t = 2$)

Supongamos que OptiBot instala un rodamiento nuevo en $t = 0$, es decir, $\mu_0 = [1.0,\ 0.0,\ 0.0]$. Vamos a calcular a mano qué ocurrirá en los turnos $t = 1$ y $t = 2$.

Paso 1: ¿Qué pasa tras el primer turno ($t = 1$)? Multiplicamos el vector μ_0 por la matriz P:

$$\begin{aligned}\mu_1 = \mu_0 P &= [1.0,\ 0.0,\ 0.0]\begin{pmatrix}0.70 & 0.20 & 0.10\\ 0.10 & 0.60 & 0.30\\ 0.10 & 0.00 & 0.90\end{pmatrix}\\ &= [(1.0\times 0.70),\ (1.0\times 0.20),\ (1.0\times 0.10)]\\ &= [\mathbf{0.70},\ \mathbf{0.20},\ \mathbf{0.10}]\end{aligned}$$

Al finalizar el primer turno, hay un $70\,\%$ de probabilidad de que la pieza siga nueva, un $20\,\%$ de que esté desgastada y un $10\,\%$ de que haya sufrido una avería temprana.

Paso 2: ¿Qué pasa tras el segundo turno ($t = 2$)? Tomamos $\mu_1 = [0.70,\ 0.20,\ 0.10]$ y lo multiplicamos nuevamente por la matriz P:

$$\mu_2 = \mu_1 P = [0.70,\ 0.20,\ 0.10] \begin{pmatrix} 0.70 & 0.20 & 0.10 \\ 0.10 & 0.60 & 0.30 \\ 0.10 & 0.00 & 0.90 \end{pmatrix}$$

Calculamos cada estado final sumando las probabilidades de todos los caminos posibles (Ley de Probabilidad Total):

- **Estar Nuevo en $t = 2$ (N):**

$$\begin{aligned} P(S_2 = N) &= (0.70 \times 0.70) + (0.20 \times 0.10) + (0.10 \times 0.10) \\ &= 0.49 + 0.02 + 0.01 = \mathbf{0.52} \quad (52\,\%) \end{aligned}$$

- **Estar Desgastado en $t = 2$ (D):**

$$\begin{aligned} P(S_2 = D) &= (0.70 \times 0.20) + (0.20 \times 0.60) + (0.10 \times 0.00) \\ &= 0.14 + 0.12 + 0.00 = \mathbf{0.26} \quad (26\,\%) \end{aligned}$$

- **Estar Averiado en $t = 2$ (A):**

$$\begin{aligned} P(S_2 = A) &= (0.70 \times 0.10) + (0.20 \times 0.30) + (0.10 \times 0.90) \\ &= 0.07 + 0.06 + 0.09 = \mathbf{0.22} \quad (22\,\%) \end{aligned}$$

El vector resultante es $\mu_2 = [0.52,\ 0.26,\ 0.22]$ (cuya suma es $0.52 + 0.26 + 0.22 = 1.0$).

Enfoque alternativo directo: Cálculo con $\mu_2 = \mu_0 P^2$ En lugar de propagar el sistema paso a paso ($\mu_1 = \mu_0 P$ y luego $\mu_2 = \mu_1 P$), podemos calcular directamente la matriz de transición a 2 pasos elevando la matriz al cuadrado ($P^2 = P \times P$):

$$\begin{aligned} P^2 = P \times P &= \begin{pmatrix} 0.70 & 0.20 & 0.10 \\ 0.10 & 0.60 & 0.30 \\ 0.10 & 0.00 & 0.90 \end{pmatrix} \begin{pmatrix} 0.70 & 0.20 & 0.10 \\ 0.10 & 0.60 & 0.30 \\ 0.10 & 0.00 & 0.90 \end{pmatrix} \\ &= \begin{pmatrix} 0.52 & 0.26 & 0.22 \\ 0.16 & 0.38 & 0.46 \\ 0.16 & 0.02 & 0.82 \end{pmatrix} \end{aligned}$$

Multiplicando ahora el vector inicial μ_0 directamente por P^2:

$$\begin{aligned} \mu_2 = \mu_0 P^2 &= [1.0,\ 0.0,\ 0.0] \begin{pmatrix} 0.52 & 0.26 & 0.22 \\ 0.16 & 0.38 & 0.46 \\ 0.16 & 0.02 & 0.82 \end{pmatrix} \\ &= [(1.0 \times 0.52),\ (1.0 \times 0.26),\ (1.0 \times 0.22)] \\ &= [\mathbf{0.52},\ \mathbf{0.26},\ \mathbf{0.22}] \end{aligned}$$

Como se comprueba, **ambos caminos son matemáticamente equivalentes y conducen al mismo resultado exacto**:

- **Cálculo paso a paso:** $\mu_2 = \mu_1 P = [0.52,\ 0.26,\ 0.22]$.
- **Cálculo directo por potencia:** $\mu_2 = \mu_0 P^2 = [0.52,\ 0.26,\ 0.22]$.

Conclusión didáctica: En tan solo dos turnos, la probabilidad de conservar la pieza en óptimas condiciones se desplomó prácticamente a la mitad ($52\,\%$), y el riesgo de que la cinta transportadora esté fuera de servicio ya ascendió al $22\,\%$.

3.5 Distribución Estacionaria: Multiplicando la Matriz Hasta el Infinito

Ahora viene la pregunta más importante para la ingeniería y la toma de decisiones: ¿qué pasará si la fábrica trabaja durante semanas, meses o años ($k \to \infty$)? ¿Las probabilidades seguirán cambiando para siempre o llegarán a un punto de equilibrio estable?

3.5.1 El Gran Descubrimiento: Multiplicar la Matriz por Sí Misma (P^k)

Para responder esto, hagamos un experimento muy revelador: tomemos la matriz de transición P y multipliquémosla por sí misma una y otra vez ($P \times P \times P \cdots = P^k$).

Recordemos qué significa cada celda (i, j) de la matriz elevada a la potencia k:

$$\left(P^k\right)_{ij} = \text{Probabilidad de llegar al estado } j \text{ tras } k \text{ turnos, habiendo salido del estado } i. \quad (3.10)$$

Ejemplo: Multiplicación sucesiva de la matriz de OptiBot

Retomando la matriz de transición a dos pasos P^2 calculada anteriormente:

$$P^2 = \begin{pmatrix} 0.520 & 0.260 & 0.220 \\ 0.160 & 0.380 & 0.460 \\ 0.160 & 0.020 & 0.820 \end{pmatrix}$$

Observemos que en $t = 2$ las tres filas todavía son muy distintas entre sí. ¿Por qué? Porque el sistema todavía recuerda de dónde vino: si empezó averiado, la probabilidad de seguir roto sigue siendo muy alta ($82\,\%$).

Multiplicaciones sucesivas ($P^3, P^5, P^{10}, P^{20}, P^{50}$) ¿Qué sucede si seguimos multiplicando la matriz por sí misma sucesivamente en la computadora?

$$P^3 = P^2 \cdot P = \begin{pmatrix} 0.4120 & 0.2600 & 0.3280 \\ 0.1960 & 0.2600 & 0.5440 \\ 0.1960 & 0.0440 & 0.7600 \end{pmatrix}$$

$$P^5 = P^3 \cdot P^2 = \begin{pmatrix} 0.3083 & 0.2125 & 0.4792 \\ 0.2306 & 0.1606 & 0.6088 \\ 0.2306 & 0.0829 & 0.6866 \end{pmatrix}$$

$$P^{10} = P^5 \cdot P^5 = \begin{pmatrix} 0.2545 & 0.1394 & 0.6061 \\ 0.2485 & 0.1253 & 0.6263 \\ 0.2485 & 0.1192 & 0.6323 \end{pmatrix}$$

$$P^{20} = P^{10} \cdot P^{10} = \begin{pmatrix} 0.2500 & 0.1252 & 0.6248 \\ 0.2500 & 0.1250 & 0.6250 \\ 0.2500 & 0.1249 & 0.6251 \end{pmatrix}$$

$$P^{50} \approx \begin{pmatrix} \mathbf{0.2500} & \mathbf{0.1250} & \mathbf{0.6250} \\ \mathbf{0.2500} & \mathbf{0.1250} & \mathbf{0.6250} \\ \mathbf{0.2500} & \mathbf{0.1250} & \mathbf{0.6250} \end{pmatrix}$$

El Momento «¡Eureka!»: ¿Qué Significa este Resultado?

Mire con detenimiento la matriz P^{50}. ¡Las tres filas se han vuelto **exactamente idénticas**! Esto revela dos propiedades matemáticas y prácticas asombrosas:

1. **Pérdida absoluta de la memoria de inicio:** A largo plazo, **no importa en absoluto cómo comenzó la máquina**. Ya sea que haya empezado nueva de fábrica ($\mu_0 = [1, 0, 0]$), desgastada ($\mu_0 = [0, 1, 0]$) o completamente rota ($\mu_0 = [0, 0, 1]$), después de 50 turnos la probabilidad de encontrarla en cada estado converge exactamente a los mismos porcentajes:

   - Si partimos de pieza nueva ($\mu_0 = [1, 0, 0]$):

   $$\mu_{50} = [1, 0, 0]P^{50} \approx [0.250,\ 0.125,\ 0.625]$$

   - Si partimos de pieza rota ($\mu_0 = [0, 0, 1]$):

   $$\mu_{50} = [0, 0, 1]P^{50} \approx [0.250,\ 0.125,\ 0.625]$$

   - Si partimos de cualquier mezcla arbitraria $\mu_0 = [p_N, p_D, p_A]$ (con $p_N + p_D + p_A = 1$):

   $$\mu_\infty = [p_N, p_D, p_A] \begin{pmatrix} 0.250 & 0.125 & 0.625 \\ 0.250 & 0.125 & 0.625 \\ 0.250 & 0.125 & 0.625 \end{pmatrix} = [\mathbf{0.250},\ \mathbf{0.125},\ \mathbf{0.625}]$$

2. **La Distribución Estacionaria (π^*):** Esa fila única a la que converge la matriz es el vector de **distribución estacionaria** $\pi^* = [\pi^*_N, \pi^*_D, \pi^*_A]$:

   $$\pi^* = [0.250,\ 0.125,\ 0.625]$$

   Esto significa que, en el largo plazo:

   - $\pi^*_N = \mathbf{25.0}\,\%$ del tiempo total estará en estado **Nuevo**.
   - $\pi^*_D = \mathbf{12.5}\,\%$ del tiempo total estará en estado **Desgastado**.
   - $\pi^*_A = \mathbf{62.5}\,\%$ del tiempo total estará en estado **Averiado**.

La Condición de Equilibrio: $\pi^* P = \pi^*$

Una vez que el sistema se encuentra en la distribución estacionaria, multiplicar nuevamente por la matriz P ya no cambia nada: el sistema ha alcanzado un balance perfecto entre lo que entra y lo que sale de cada estado:

$$\pi^* \mathbf{P} = \pi^* \quad \text{sujeto a} \quad \sum_{i=1}^{n} \pi^*_i = 1 \tag{3.11}$$

Verificación algebraica manual de la igualdad: Comprobemos que nuestro vector $\pi^* = [0.250,\ 0.125,\ 0.625]$ satisface exactamente la fórmula:

$$\begin{aligned}
\text{Para } N: &\quad 0.70(0.250) + 0.10(0.125) + 0.10(0.625) = 0.175 + 0.0125 + 0.0625 = \mathbf{0.250} \quad \checkmark \\
\text{Para } D: &\quad 0.20(0.250) + 0.60(0.125) + 0.00(0.625) = 0.050 + 0.075 + 0.000 = \mathbf{0.125} \quad \checkmark \\
\text{Para } A: &\quad 0.10(0.250) + 0.30(0.125) + 0.90(0.625) = 0.025 + 0.0375 + 0.5625 = \mathbf{0.625} \quad \checkmark
\end{aligned}$$

¿Por qué ocurre esta convergencia? Para que una cadena de Markov tenga una única distribución estacionaria independiente del estado inicial, debe ser **ergódica**:

1. **Irreducible:** Es posible viajar de cualquier estado a cualquier otro con el tiempo (no

hay trampas de las que sea imposible escapar).

2. **Aperiódica:** El sistema no oscila en un ciclo rígido como las manecillas de un reloj ($A \to B \to A \to B$).

Conclusión para el sistema de OptiBot: Si dejamos que el almacén funcione en piloto automático sin una estrategia inteligente de mantenimiento, la cinta transportadora pasará un alarmante $62.5\,\%$ **del tiempo fuera de servicio por averías**. Este resultado matemático demuestra con total claridad por qué necesitamos agentes inteligentes: OptiBot no puede limitarse a observar cómo se rompen las máquinas; debe aprender a **tomar acciones óptimas** en cada estado para maximizar la productividad.

3.6 Implementación en Python: Proyección y Distribución Estacionaria

Para experimentar de forma interactiva con el cálculo de cadenas de Markov, proyecciones temporales y cálculo de la distribución estacionaria mediante álgebra matricial, el código completo se encuentra disponible en el cuaderno Jupyter:

`notebooks/cap02_markov.ipynb`

En este cuaderno interactivo se definen las matrices estocásticas de transición con NumPy, se proyecta la evolución del estado de las cintas transportadoras a corto plazo ($t = 1, t = 2$), se ilustra la convergencia asintótica mediante potencias matriciales sucesivas (P^k) y se calcula de forma exacta la distribución estacionaria (π^*) utilizando descomposición en autovectores y autovalores.

Las Cadenas de Markov nos permiten describir y predecir cómo cambia un entorno estocástico con el tiempo. Sin embargo, en este modelo el agente es un simple espectador pasivo: las probabilidades ocurren por sí solas y no hay posibilidad de intervenir.
En el siguiente capítulo daremos el gran salto: uniremos las **acciones y recompensas** del Capítulo 2 con los **estados dinámicos** de este capítulo. Esta combinación da lugar a la estructura reina del Aprendizaje por Refuerzo: el **Proceso de Decisión de Markov (MDP)**, el cual estudiaremos en el Capítulo 4.

4

Procesos de Decisión de Markov y Ecuaciones de Bellman

4.1 La Unificación de Estados, Acciones y Recompensas

Los capítulos precedentes han desarrollado de forma independiente dos dimensiones complementarias de la toma de decisiones:

- En el Capítulo 2, se abordó cómo elegir entre diferentes **acciones** evaluando beneficios numéricos inmediatos, pero en un entorno sin movimiento ni **estados**.
- En el Capítulo 3, se analizó la evolución temporal de **estados** físicos en el almacén, pero en un régimen puramente pasivo donde el sistema cambiaba sin que un agente pudiera intervenir con **acciones**.

El **Proceso de Decisión de Markov** (*Markov Decision Process*, o MDP) representa la síntesis definitiva de todo el aprendizaje: en cada instante de tiempo, el agente observa en qué **estado** se encuentra, evalúa las alternativas disponibles, elige una **acción** deliberada, el entorno cambia hacia un nuevo estado y devuelve una señal evaluativa formal llamada **recompensa**.

4.1.1 La Hipótesis de la Recompensa

¿Cómo se le comunica a un robot o algoritmo qué es lo que se espera que logre? En el Aprendizaje por Refuerzo, todo objetivo se formaliza a través de una señal escalar denominada **recompensa** (R_t).

Este principio se conoce como la **Hipótesis de la Recompensa**:

> *«Todo lo que entendemos por metas y propósitos puede formalizarse adecuadamente como la maximización del valor esperado de la suma acumulada de una señal numérica escalar (la recompensa).»*

La recompensa no le dice al agente qué pasos específicos debe ejecutar (no es una instrucción); únicamente le comunica qué tan deseable es el resultado alcanzado. Con esta señal, el agente debe descubrir la secuencia de decisiones óptima.

En esta fase del desarrollo, OptiBot opera como un robot móvil autónomo encargado de completar el **ciclo de despacho de pedidos** a lo largo de cuatro zonas operativas del almacén:

1. **Muelle de Carga (S_1):** Zona inicial de recepción. OptiBot debe decidir cómo cargar y despachar el pedido.
2. **Pasillo Central (S_2):** Zona de navegación y transporte. OptiBot debe seleccionar el modo de conducción para trasladar el paquete.
3. **Bahía de Entrega (S_3):** Área final de descarga del producto antes de salir hacia el cliente.
4. **Fin de Turno / Misión Cumplida (S_4):** Estado terminal absorbente donde la entrega se ha completado exitosamente y concluye el episodio ($V(S_4) = 0$).

En cada uno de los estados activos (S_1, S_2, S_3), OptiBot dispone de **dos acciones alternativas** que alteran sus probabilidades de avance y su consumo energético.

4.2 Definición Formal de un MDP: La Tupla de Siete Elementos

A diferencia de las Cadenas de Markov, que modelan sistemas pasivos únicamente mediante estados y transiciones, un Proceso de Decisión de Markov permite formalizar la toma activa de decisiones mediante una tupla de siete elementos:

$$\mathcal{M} = \langle \mathcal{S},\, \mathcal{A},\, \mathcal{P},\, \mathcal{R},\, \gamma,\, \mu,\, \pi \rangle \tag{4.1}$$

cuyos componentes se definen como:

- $\mathcal{S}$ (**Espacio de Estados**): Conjunto de situaciones o posiciones en las cuales puede encontrarse el sistema ($\mathcal{S} = \{S_1, S_2, S_3, S_4\}$).
- $\mathcal{A}$ (**Espacio de Acciones**): Conjunto de decisiones de control ejecutables por el agente en cada estado.
- $\mathcal{P}$: Función de dinámica de transición que describe la probabilidad condicional de que el sistema pase al estado sucesor s' habiendo tomado la acción a en el estado actual s:

$$P(s' \mid s, a) \doteq P(S_{t+1} = s' \mid S_t = s, A_t = a) \tag{4.2}$$

- $\mathcal{R}$: Función de recompensa que asigna un valor escalar esperado a cada transición ejecutada bajo el par estado-acción:

$$R(s, a, s') \doteq \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s'] \tag{4.3}$$

- $\gamma \in [0, 1)$ (**Factor de Descuento**): Ponderador temporal que regula la importancia relativa de las recompensas futuras frente a las inmediatas.
- μ: Distribución inicial de estados, que asigna la probabilidad de comenzar en cada estado del sistema (en OptiBot: $\mu = [1, 0, 0, 0]$, pues siempre parte del Muelle).
- π: Política del agente, que define su estrategia asignando a cada estado una distribución de probabilidad sobre las acciones elegibles: $\pi(a \mid s) \doteq P(A_t = a \mid S_t = s)$.

4.2.1 El Grafo del MDP: Estados, Acciones y Recompensas

La incorporación de acciones transforma la simple cadena de Markov en un grafo de decisión. La Figura 4.1 ilustra el modelo de OptiBot:

- Los **círculos continuos** representan los estados activos (S_1, S_2, S_3), y el círculo doble representa el estado terminal (S_4).
- Las **flechas continuas** indican las acciones disponibles en cada estado.
- Las **flechas discontinuas** representan las transiciones estocásticas del entorno hacia el estado sucesor, rotuladas con el par $(p,\ r)$: la probabilidad de transición $p(s' \mid s, a)$ y la recompensa escalar obtenida r.

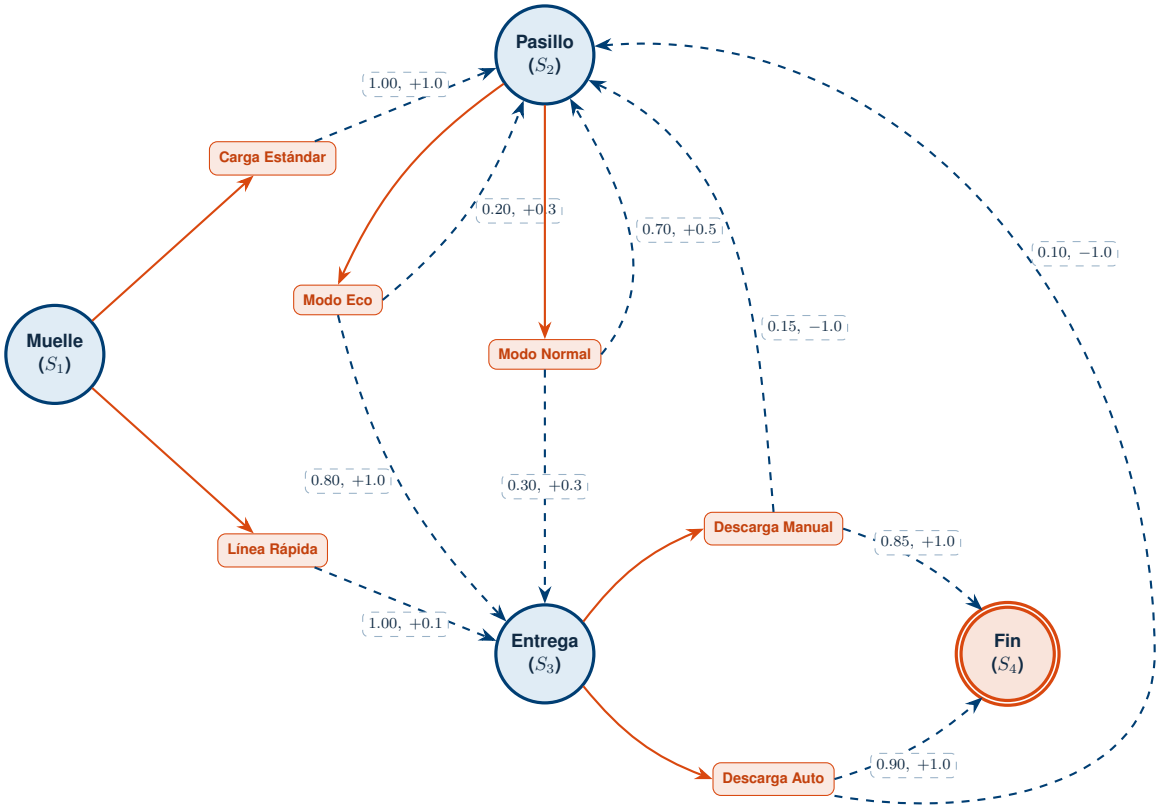


Figura 4.1: Grafo de transición del Proceso de Decisión de Markov para el ciclo logístico de OptiBot. Las flechas continuas conectan cada estado con sus acciones disponibles. Las flechas discontinuas representan las probabilidades de transición y recompensas $(p,\ r)$ resultantes hacia los estados sucesores.

4.2.2 Representación Matricial del MDP

El MDP de la Figura 4.1 se puede representar de forma matricial definiendo una **matriz de transición** (MT_a) y una **matriz de recompensa** (R_a) para cada una de las acciones disponibles:

1. Matrices de Transición por Acción (MT_a)

Cada celda (i, j) indica la probabilidad de saltar del estado S_i al estado S_j si el agente ejecuta dicha acción:

$$MT_{\text{Estándar}} = \begin{pmatrix} 0 & 1.00 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \qquad MT_{\text{Rápida}} = \begin{pmatrix} 0 & 0 & 1.00 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$$MT_{\text{Eco}} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0.20 & 0.80 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \qquad MT_{\text{Normal}} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0.70 & 0.30 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$$MT_{\text{Manual}} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0.15 & 0 & 0.85 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \qquad MT_{\text{Auto}} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0.10 & 0 & 0.90 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

2. Matrices de Recompensa por Acción (R_a)

De forma análoga, cada celda (i, j) define la recompensa obtenida al transitar de S_i a S_j bajo esa acción:

$$R_{\text{Estándar}} = \begin{pmatrix} 0 & 1.00 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \qquad R_{\text{Rápida}} = \begin{pmatrix} 0 & 0 & 0.10 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$$R_{\text{Eco}} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0.30 & 1.00 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \qquad R_{\text{Normal}} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0.50 & 0.30 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$$R_{\text{Manual}} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & -1.00 & 0 & 1.00 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \qquad R_{\text{Auto}} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & -1.00 & 0 & 1.00 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

4.2.3 Tabla de Parámetros del MDP

El Cuadro 4.1 resume todas las dinámicas y recompensas del entorno en una estructura tabular compacta:

4.2.4 El Retorno Descontado y la Preferencia Temporal

El objetivo de OptiBot no es obtener un premio aislado, sino maximizar el **retorno acumulado descontado** G_t:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{4.4}$$

El parámetro $\gamma \in [0, 1)$ pondera la preferencia temporal: valores cercanos a 0 generan agentes miopes centrados en la gratificación inmediata, mientras que valores próximos a 1 promueven agentes previsores y pacientes. Para este problema adoptamos $\gamma = 0.90$.

Estado Inicial (s)	Acción (a)	Probabilidad (p)	Recompensa (r)	Nuevo Estado (s')
Muelle (S_1)	Carga Estándar	1.00	$+1.00$	Pasillo (S_2)
Muelle (S_1)	Línea Rápida	1.00	$+0.10$	Entrega (S_3)
Pasillo (S_2)	Modo Eco	0.20	$+0.30$	Pasillo (S_2)
Pasillo (S_2)	Modo Eco	0.80	$+1.00$	Entrega (S_3)
Pasillo (S_2)	Modo Normal	0.70	$+0.50$	Pasillo (S_2)
Pasillo (S_2)	Modo Normal	0.30	$+0.30$	Entrega (S_3)
Entrega (S_3)	Descarga Manual	0.15	-1.00	Pasillo (S_2)
Entrega (S_3)	Descarga Manual	0.85	$+1.00$	Fin (S_4)
Entrega (S_3)	Descarga Auto	0.10	-1.00	Pasillo (S_2)
Entrega (S_3)	Descarga Auto	0.90	$+1.00$	Fin (S_4)

Cuadro 4.1: Parámetros de transición y recompensas de cada acción del MDP de OptiBot.

4.3 Políticas y Funciones de Valor

Una **política** π formaliza el comportamiento del agente. Para evaluar y comparar políticas, se introducen dos funciones de valor fundamentales:

1. **Función de Valor de Estado** $V^\pi(s)$**:** Valor esperado del retorno al comenzar en el estado s y seguir la política π:
$$V^\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s] \tag{4.5}$$
2. **Función de Valor de Acción** $Q^\pi(s,a)$**:** Valor esperado del retorno al ejecutar la acción a en el estado s, siguiendo π en adelante:
$$Q^\pi(s,a) \doteq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] \tag{4.6}$$

4.4 La Ecuación de Expectativa de Bellman

La genialidad de Richard Bellman consistió en demostrar que el valor de un estado no requiere simular infinitos pasos futuros: se expresa como una relación recursiva de dos términos:

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) \Big[r + \gamma V^\pi(s') \Big] \tag{4.7}$$

Es decir: el valor de estar hoy en s es igual a la recompensa inmediata promedio que se recibe al dar el paso (r), más el valor futuro descontado de dónde aterrice el agente ($\gamma V(s')$).

4.5 Evaluación Iterativa de Políticas y Determinación de la Política Óptima

Para resolver el MDP y descubrir la política óptima π^* que indique qué acción debe ejecutarse en cada estado, se emplea el algoritmo clásico de **Evaluación Iterativa de Políticas** / **Iteración de Valores** formulado por Sutton y Barto:

Evaluación Iterativa de Políticas, para estimar $V \approx v_\pi$

Entrada: π, la política a ser evaluada.
Parámetro del algoritmo: un umbral pequeño $\theta > 0$ que determina la precisión de la estimación.
Inicializar: $V(s)$, para todo $s \in \mathcal{S}^+$, arbitrariamente excepto que $V(\text{terminal}) = 0$.

Bucle:
$\Delta \leftarrow 0$
Bucle para cada $s \in \mathcal{S}$:
$v \leftarrow V(s)$
$V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)\Big[r + \gamma V(s')\Big]$
$\Delta \leftarrow \max\big(\Delta, |v - V(s)|\big)$
hasta que $\Delta < \theta$

4.5.1 Resolución Manual Paso a Paso: Aplicación en OptiBot

Consideramos un umbral de convergencia $\theta = 0.01$ y un factor de descuento $\gamma = 0.90$.

Paso 1: Inicialización. Como primer paso, se establece la función valor $V(s)$ de cada estado en algún valor arbitrario. Por simplicidad, inicializamos todos los valores en cero con la condición estricta de que el estado terminal siempre vale cero:

Estado (s)	$V_0(s)$
Muelle (S_1)	0.00
Pasillo (S_2)	0.00
Entrega (S_3)	0.00
Fin (S_4)	0.00 (Terminal)

Cuadro 4.2: Función de valor inicial de cada estado del MDP de OptiBot.

Paso 2: Primera Iteración ($k = 1$). En cada iteración, el parámetro de variación Δ se inicializa en cero. Para cada estado, se calcula el valor de cada acción posible evaluando el término:

$$p(s', r \mid s, a)\Big[r + \gamma V_0(s')\Big] \tag{4.8}$$

y se suman los resultados para obtener el rendimiento acumulado Σ de dicha acción. Luego, se selecciona el valor más alto $(\max)$ entre las acciones disponibles y se asigna como el nuevo valor $V(s)$ de ese estado.

El Cuadro 4.3 muestra el desglose aritmético completo de la primera iteración:

Como $\Delta = 1.0000 \geq 0.01$, el algoritmo no se detiene y avanza a la siguiente iteración utilizando el nuevo vector de valores $[1.0000,\ 0.8600,\ 0.8000]$.

Paso 3: Iteración Final y Convergencia. El ciclo de actualización se repite de forma progresiva. En cada pasada, los valores futuros descubiertos se propagan hacia atrás a través del factor de descuento $\gamma = 0.90$. Tras 7 iteraciones, la variación máxima se reduce a $\Delta = 0.009071$, cumpliendo con éxito la condición de parada $\Delta < 0.01$.

El Cuadro 4.4 documenta la iteración final del algoritmo:

s	a	s'	r	$p(s' \mid s,a)$	$V_0(s')$	$p(s' \mid s,a)[r+\gamma V_0(s')]$	Σ
Muelle	Carga Estándar	Pasillo	1.00	1.00	0.0000	1.000000	**1.000000**
Muelle	Línea Rápida	Entrega	0.10	1.00	0.0000	0.100000	0.100000
Pasillo	Modo Eco	Pasillo	0.30	0.20	0.0000	0.060000	0.060000
Pasillo	Modo Eco	Entrega	1.00	0.80	0.0000	0.800000	**0.860000**
Pasillo	Modo Normal	Pasillo	0.50	0.70	0.0000	0.350000	0.350000
Pasillo	Modo Normal	Entrega	0.30	0.30	0.0000	0.090000	0.440000
Entrega	Descarga Manual	Pasillo	−1.00	0.15	0.0000	−0.150000	−0.150000
Entrega	Descarga Manual	Fin	1.00	0.85	0.0000	0.850000	0.700000
Entrega	Descarga Auto	Pasillo	−1.00	0.10	0.0000	−0.100000	−0.100000
Entrega	Descarga Auto	Fin	1.00	0.90	0.0000	0.900000	**0.800000**

Actualización de valores de $V(s)$:

s	$V(s)_{\text{anterior}}$	$V(s)_{\text{nuevo}}$	Diferencia	$\Delta = \max(\Delta, \text{diferencia})$
Muelle (S_1)	0.000000	1.000000	1.000000	1.000000
Pasillo (S_2)	0.000000	0.860000	0.860000	1.000000
Entrega (S_3)	0.000000	0.800000	0.800000	1.000000

Cuadro 4.3: Primera iteración del algoritmo de evaluación iterativa para OptiBot.

s	a	s'	r	$p(s' \mid s,a)$	$V_6(s')$	$p(s' \mid s,a)[r+\gamma V_6(s')]$	Σ
Muelle	Carga Estándar	Pasillo	1.00	1.00	1.895925	2.706333	**2.706333**
Muelle	Línea Rápida	Entrega	0.10	1.00	0.969695	0.972726	0.972726
Pasillo	Modo Eco	Pasillo	0.30	0.20	1.895925	0.401267	0.401267
Pasillo	Modo Eco	Entrega	1.00	0.80	0.969695	1.498180	**1.899447**
Pasillo	Modo Normal	Pasillo	0.50	0.70	1.895925	1.544433	1.544433
Pasillo	Modo Normal	Entrega	0.30	0.30	0.969695	0.351818	1.896251
Entrega	Descarga Manual	Pasillo	−1.00	0.15	1.895925	0.105950	0.105950
Entrega	Descarga Manual	Fin	1.00	0.85	0.000000	0.850000	0.955950
Entrega	Descarga Auto	Pasillo	−1.00	0.10	1.895925	0.070633	0.070633
Entrega	Descarga Auto	Fin	1.00	0.90	0.000000	0.900000	**0.970633**

Actualización de valores de $V(s)$:

s	$V(s)_{\text{anterior}}$	$V(s)_{\text{nuevo}}$	Diferencia	$\Delta = \max(\Delta, \text{diferencia})$
Muelle (S_1)	2.697262	2.706333	0.009071	0.009071
Pasillo (S_2)	1.895925	1.899447	0.003522	0.009071
Entrega (S_3)	0.969695	0.970633	0.000938	0.009071

Cuadro 4.4: Iteración final (Iteración 7) del algoritmo con convergencia garantizada ($\Delta = 0.009071 < 0.01$).

4.5.2 Deducción de la Política Óptima

Una vez que el algoritmo converge, la política óptima π^* se obtiene identificando la acción que obtuvo el valor más alto en la columna Σ para cada estado del sistema:

$$\pi^*(s) = \begin{cases} (Muelle, \text{ Carga Estándar}) & \text{con } V^*(S_1) \approx 2.71 \\ (Pasillo, \text{ Modo Eco}) & \text{con } V^*(S_2) \approx 1.90 \\ (Entrega, \text{ Descarga Automática}) & \text{con } V^*(S_3) \approx 0.97 \end{cases} \tag{4.9}$$

Interpretación Operativa de la Política Óptima

A diferencia de un modelo con incentivos mal calibrados, este resultado tiene una coherencia lógica absoluta en el contexto operativo del almacén:

1. **En el Muelle (S_1):** OptiBot prefiere la **Carga Estándar** sobre la Línea Rápida (2.71 vs 0.97). Aunque la línea rápida lo teletransporta directamente a la entrega, el costo de activación penaliza fuertemente la ganancia inicial ($r = 0.10$ vs 1.00), haciendo que el despacho normal sea económicamente superior a largo plazo.

2. **En el Pasillo (S_2):** OptiBot selecciona el **Modo Eco** frente al Modo Normal (1.90 vs 1.89). A pesar de que el modo normal ofrece una recompensa inmediata mayor por demorarse (0.50 vs 0.30), el Modo Eco ofrece una probabilidad de avance del $80\,\%$ hacia la bahía de entrega frente a solo el $30\,\%$ del modo normal. El agente prefiere llegar antes a la meta que demorarse cobrando pequeñas recompensas intermedias.

3. **En la Bahía de Entrega (S_3):** OptiBot elige la **Descarga Automática** (0.97 vs 0.95). Al reducir el riesgo de atascos al $10\,\%$ (frente al $15\,\%$ del método manual), asegura con un $90\,\%$ de probabilidad el éxito final del pedido.

4. **Evolución decreciente hacia la meta ($2.71 \to 1.90 \to 0.97 \to 0$):** El valor de cada estado representa el **retorno restante esperado** hasta culminar el turno. Cuanto más cerca está el robot de completar la entrega, menos etapas le quedan por recorrer y menor es el retorno futuro acumulable, alcanzando exactamente cero en el estado terminal Fin.

4.6 Implementación en Python: Iteración de Valor y Ecuaciones de Bellman

Para experimentar de forma interactiva con la resolución del MDP de OptiBot y el algoritmo de Iteración de Valor (*Value Iteration*), el código completo se encuentra disponible en el cuaderno Jupyter:

`notebooks/cap03_mdp_bellman.ipynb`

En este cuaderno interactivo se definen las funciones de transición y recompensa esperada del almacén, se implementa el bucle de convergencia de Bellman iterando hasta satisfacer el umbral de tolerancia $\theta = 0.01$, y se extrae automáticamente la política óptima $\pi^*(s)$. El algoritmo converge en 7 iteraciones reproduciendo con exactitud los valores calculados teóricamente: $V^*(S_1) = 2.7095$, $V^*(S_2) = 1.9008$ y $V^*(S_3) = 0.9710$, confirmando que la política óptima consiste en utilizar Carga Estándar en el muelle, Modo Eco en el pasillo y Descarga Automática en la bahía de entrega.

El puente hacia el siguiente capítulo: En este capítulo hemos definido rigurosamente qué es un Proceso de Decisión de Markov y cómo evaluar sus funciones de valor mediante las Ecuaciones de Bellman bajo el supuesto de conocer el modelo analítico del entorno. En el Capítulo 5, abordaremos el desafío fundamental del **Aprendizaje por Refuerzo Tradicional**: ¿cómo puede OptiBot aprender la política óptima sin disponer de planos ni modelos analíticos de probabilidad? Introduciremos el aprendizaje libre de modelo mediante Diferencia Temporal y deduciremos en detalle el algoritmo fundamental **Q-Learning**.

5

Aprendizaje por Refuerzo Tradicional: Algoritmo Q-Learning

5.1 De las Ecuaciones Analíticas al Aprendizaje sin Modelo (Model-Free)

En el Capítulo 4, formalizamos los Procesos de Decisión de Markov (MDP) y descubrimos cómo calcular la política óptima mediante la **Ecuación de Bellman** y el algoritmo de evaluación iterativa. Sin embargo, en aquel escenario asumíamos una condición ideal: conocíamos de antemano el **modelo completo del entorno**, es decir, las tablas exactas de probabilidad de transición $\mathcal{P}^a_{ss'}$ y las funciones analíticas de recompensa $\mathcal{R}(s, a)$.

En el mundo físico real, esa suposición casi nunca se cumple:

- Un robot móvil no dispone de un manual con las probabilidades exactas de que sus ruedas patinen o de que una compuerta se trabe.
- Modelar matemáticamente cada interacción física, fricción y obstáculo imprevisto resulta inviable.
- El agente debe enfrentarse a lo desconocido: solo puede descubrir qué acciones son seguras y cuáles son perjudiciales **interactuando directamente con el entorno mediante prueba y error**.

Este es el territorio del **Aprendizaje por Refuerzo Tradicional**: el paradigma **libre de modelo** (*Model-Free RL*). El agente no asume conocer cómo funciona el mundo en teoría; en su lugar, **aprende observando las consecuencias reales de sus decisiones**, recopilando experiencias paso a paso y corrigiendo su estrategia a través del algoritmo fundamental de Watkins (1989): **Q-Learning**.

5.2 El Laberinto de OptiBot: Un Almacén en Cuadrícula de 3×3

Para comprender el funcionamiento de Q-Learning sin perderse en fórmulas abstractas, consideremos el problema arquetípico del aprendizaje por refuerzo: el **almacén inteligente modelado como una cuadrícula o laberinto de** 3×3 (Figura 5.1).

Las reglas de este entorno son transparentes y directas:

1. **Espacio de Estados ($\mathcal{S}$):** Hay 9 casillas numeradas del S_1 al S_9.

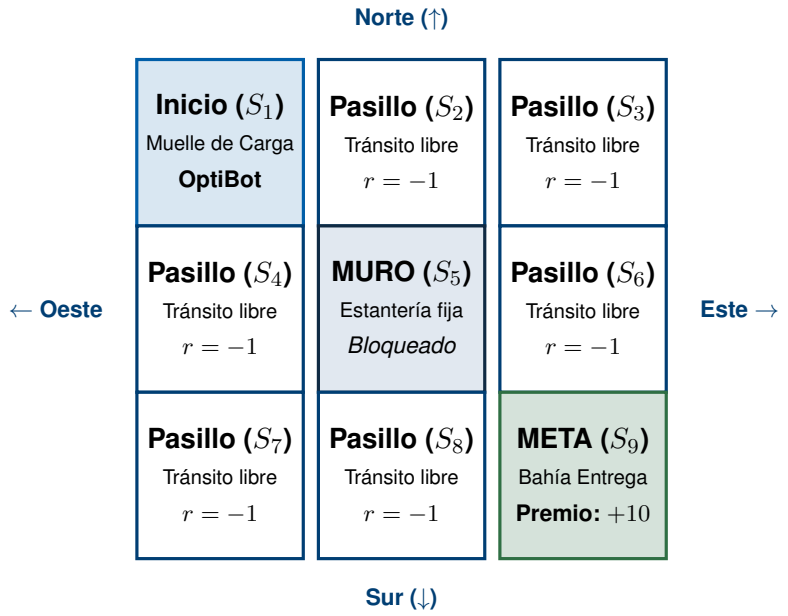


Figura 5.1: El entorno del laberinto logístico de OptiBot (3×3). El robot parte del Muelle (S_1) y debe alcanzar la Meta (S_9) esquivando la estantería central bloqueada (S_5). Cada movimiento ordinario descuenta -1 de recompensa, incentivando descubrir la ruta más rápida.

- S_1 es la casilla de **Inicio** donde parte OptiBot.
- S_5 es un **Muro** (una estantería fija impenetrable): el robot no puede ingresar en ella.
- S_9 es la **Meta** (Bahía de Entrega final): es un estado terminal que concede una bonificación de $+10$ puntos al llegar y concluye el episodio.

2. **Acciones disponibles ($\mathcal{A}$):** En cualquier casilla, OptiBot puede intentar moverse en cuatro direcciones cardinales:

$$\mathcal{A} = \{\text{Norte } (\uparrow), \text{ Sur } (\downarrow), \text{ Este } (\rightarrow), \text{ Oeste } (\leftarrow)\}$$

3. **Dinámica, Paredes y Salida del Laberinto:**
   - **Muro Central (S_5):** Es una estantería fija impenetrable. Si OptiBot intenta ingresar a S_5, rebota contra el obstáculo y permanece en su misma casilla, sufriendo la penalización normal de paso ($R = -1.0$).
   - **Límites del Laberinto (Game Over):** Si OptiBot intenta moverse hacia fuera de los límites perimetrales de la cuadrícula (por ejemplo, avanzar al Norte o al Oeste desde S_1), el robot cae fuera del recinto de trabajo. En este caso, el entorno le aplica una severa penalización de $R = -10.0$ y el episodio concluye de inmediato (*Game Over*, estado terminal con valor futuro nulo).
4. **Señal de Recompensa (R):**
   - Cada paso ordinario por los pasillos cuesta $R = -1.0$ (penalización por tiempo y consumo de batería).
   - Intentar salirse de los límites del laberinto cuesta $R = -10.0$ y causa *Game Over* (fin de episodio con penalización severa).
   - Ingresar a la Meta (S_9) otorga una gran recompensa de $R = +10.0$ (fin de episodio exitoso).

El objetivo de OptiBot es encontrar la **secuencia más corta de movimientos** que lo conduzca a la Meta (S_9) esquivando el muro central (S_5) y acumulando la máxima puntuación posible, **sin conocer de antemano el mapa ni las recompensas**.

5.3 ¿Qué es exactamente la Tabla Q? La Memoria de Navegación

Imaginemos que OptiBot lleva consigo una libreta de notas con una cuadrícula dibujada. A esa libreta la llamamos la **Tabla Q**.

La letra **Q** proviene del término inglés *Quality* (Calidad). Cada casilla de la libreta le asigna una puntuación a la pregunta más importante del aprendizaje:

> *«Si estoy en la casilla s y doy un paso en la dirección a, ¿qué tan buena idea es ese movimiento para mi futuro?»*

5.3.1 Estructura de la Tabla Q

La Tabla Q es simplemente una matriz bidimensional:

- Las **filas** representan las casillas del almacén ($S_1, S_2, \ldots, S_9$).
- Las **columnas** representan las 4 direcciones posibles ($\uparrow, \downarrow, \rightarrow, \leftarrow$).
- El valor dentro de la celda $Q(s, a)$ es un número decimal que estima el retorno acumulado total esperado.

Estado (Casilla)	Norte ($\uparrow$)	Sur ($\downarrow$)	Este ($\rightarrow$)	Oeste ($\leftarrow$)
S_1 (Inicio)	0.00	0.00	0.00	0.00
S_2 (Pasillo)	0.00	0.00	0.00	0.00
S_3 (Pasillo)	0.00	0.00	0.00	0.00
S_4 (Pasillo)	0.00	0.00	0.00	0.00
S_5 (Muro)	—	—	—	—
S_6 (Pasillo)	0.00	0.00	0.00	0.00
S_7 (Pasillo)	0.00	0.00	0.00	0.00
S_8 (Pasillo)	0.00	0.00	0.00	0.00
S_9 (Meta)	0.00	0.00	0.00	0.00

Cuadro 5.1: La Tabla Q inicial de OptiBot. Al comenzar, todas las celdas valen cero: el robot es una "página en blanco" que desconoce dónde está la meta y dónde están los muros.

5.3.2 La Regla de Decisión: Elegir la Flecha con Mayor Puntuación

Una vez que la tabla tiene números aprendidos, la toma de decisiones es directa: el robot consulta la fila de la casilla s en la que se encuentra y selecciona la dirección con la mayor puntuación. Esta regla formaliza la **política codiciosa** (*greedy policy*):

$$\pi(s) = \arg\max_{a \in \mathcal{A}} Q(s, a) \tag{5.1}$$

En palabras sencillas, esta fórmula representa la regla de navegación de OptiBot: el operador $\arg\max$ no se queda con el puntaje numérico, sino que extrae la **acción física concreta** (Norte,

Sur, Este u Oeste) que produce el valor $Q(s, a)$ más alto en dicha casilla. ¡Nótese la enorme ventaja frente al capítulo anterior! No se requieren probabilidades de transición, ni integrales, ni sumatorias. Basta con mirar la fila y tomar el máximo.

5.4 La Ecuación de Q-Learning Explicada Paso a Paso

¿Cómo hace el robot para llenar esa tabla con números correctos a partir de la experiencia directa? Lo hace aplicando en cada paso la siguiente ecuación de actualización:

$$\underbrace{Q(S_t, A_t)}_{\text{Nuevo valor}} \leftarrow \underbrace{Q(S_t, A_t)}_{\text{Valor viejo}} + \alpha \cdot \Big[\underbrace{\overbrace{R_{t+1}}^{\text{Recompensa inmediata}} + \gamma \overbrace{\max_{a'} Q(S_{t+1}, a')}^{\text{Mejor valor del destino}}}_{\text{Realidad experimentada (Objetivo Bellman)}} - \underbrace{Q(S_t, A_t)}_{\text{Lo que creía antes}} \Big] \tag{5.2}$$

Desglosemos cada pieza de esta fórmula con total transparencia:

1. **¿Dónde estoy y qué hice? (S_t, A_t):** Es la casilla actual donde estaba el robot (S_t) y la dirección en la que decidió moverse (A_t). La celda $Q(S_t, A_t)$ es la que vamos a corregir en la tabla.
2. **La Recompensa Inmediata (R_{t+1}):** Es la señal concreta que el entorno devuelve inmediatamente tras dar el paso (por ejemplo, -1 por el costo de energía, o $+10$ si aterrizó en la Meta).
3. **La Mirada al Futuro ($\max_{a'} Q(S_{t+1}, a')$):** Cuando OptiBot llega a la nueva casilla (S_{t+1}), abre su libreta en la fila de esa nueva casilla y se pregunta:

   > *«Desde esta nueva casilla donde aterricé, ¿cuál es la mejor salida posible que promete mi tabla?»*

   Toma el número más alto de esa fila y lo multiplica por γ (el factor de descuento, típicamente 0.90), que refleja que las recompensas futuras valen un poco menos que las inmediatas.
4. **La Sorpresa o Error TD (δ_t):** Compara la realidad experimentada (la recompensa recibida más el mejor futuro prometido) contra lo que antes creía que valía esa acción:

$$\delta_t = \Big[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') \Big] - Q(S_t, A_t) \tag{5.3}$$

   - Si $\delta_t > 0$, el resultado fue **mejor de lo que esperaba** (¡sorpresa agradable!), por lo que la puntuación de esa casilla sube.
   - Si $\delta_t < 0$, el resultado fue **peor de lo que esperaba** (costo inesperado o golpe contra un muro), por lo que la puntuación baja.
5. **La Tasa de Aprendizaje (α):** Regula qué tan rápido aprende el robot (habitualmente $\alpha = 0.2$ o 0.5). Evita que el agente reaccione de forma exagerada ante un único resultado aislado, promediando suavemente las experiencias.

5.5 El Dilema: ¿Cómo Explorar el Laberinto? (Política ε-Greedy)

Si al principio todas las casillas valen 0.00, o si OptiBot descubre un camino mediocre y siempre elige la acción con el valor más alto actual, el robot se quedará atrapado repitiendo el mismo camino tonto y jamás descubrirá dónde está la Meta. Para solucionar esto, se utiliza la estrategia ε**-greedy** (*épsilon-codiciosa*):

- Con probabilidad $1-\varepsilon$ (por ejemplo, el 80 % de las veces), el robot **explota**: elige la dirección que tenga el valor Q más alto en su tabla.
- Con probabilidad ε (por ejemplo, el 20 % de las veces), el robot **explora**: tira una moneda al aire y elige una dirección completamente al azar para explorar rincones desconocidos.

Al principio del entrenamiento, se fija $\varepsilon = 1.0$ (exploración total). Conforme el robot completa cientos de misiones y la tabla se va llenando de números fiables, ε decae gradualmente hasta 0.01, pasando a un modo donde casi siempre utiliza su conocimiento experto aprendido.

5.6 Cálculo Manual Paso a Paso: Siguiendo a OptiBot en el Laberinto

Para ver cómo funciona la matemática con números exactos, sigamos el recorrido de OptiBot paso a paso con lápiz y papel.

Parámetros del ejercicio:

- Tasa de aprendizaje: $\alpha = 0.50$ (para simplificar los cálculos a números exactos).
- Factor de descuento: $\gamma = 0.90$.
- Recompensa de paso normal por pasillo: $R = -1.0$.
- Penalización por salirse del laberinto (*Game Over*): $R = -10.0$ (estado terminal con valor futuro 0).
- Recompensa en la Meta (S_9): $R = +10.0$ (estado terminal con valor futuro 0).

Inicialmente, todas las celdas de la Tabla Q valen 0.00.

Episodio 1 – Paso 1: El primer paso en el laberinto

1. OptiBot se encuentra en el Inicio (S_1).
2. Elige al azar la acción **Este (→)**.
3. Se desplaza y aterriza en la casilla S_2. Recibe una recompensa de paso $R_1 = -1$.
4. Mira la fila del estado destino S_2 en su tabla: como todo vale cero, el máximo futuro es:
$$\max_{a'} Q(S_2, a') = \max(0, 0, 0, 0) = \mathbf{0.0}$$
5. Calcula el Objetivo de Bellman:
$$\text{Objetivo} = R_1 + \gamma \max_{a'} Q(S_2, a') = -1 + (0.90 \times 0.0) = \mathbf{-1.0}$$
6. Calcula el Error TD:
$$\delta = \text{Objetivo} - Q(S_1, \rightarrow) = -1.0 - 0.0 = \mathbf{-1.0}$$
7. Actualiza la celda $Q(S_1, \rightarrow)$:
$$Q_{\text{nuevo}}(S_1, \rightarrow) = 0.0 + 0.50 \times (-1.0) = \mathbf{-0.50}$$

Interpretación: Ahora la tabla sabe que moverse al Este desde S_1 tiene un valor de -0.50.

Episodio 1 – Paso 2: Avanzando por el laberinto

1. OptiBot está en S_2 y elige moverse hacia el **Este (→)**.
2. Aterriza en la esquina superior derecha (S_3) y recibe $R_2 = -1$.
3. El valor futuro en S_3 es cero. Actualizamos:

$$Q_{\text{nuevo}}(S_2, \rightarrow) = 0.0 + 0.50 \times [-1 + 0.9(0) - 0] = \mathbf{-0.50}$$

Episodio 1 – Paso 3: Bajando hacia el sur

1. Desde S_3, el robot elige moverse hacia el **Sur (↓)**.
2. Aterriza en la casilla S_6 (a la derecha del muro S_5) y recibe $R_3 = -1$.
3. Actualizamos de forma análoga:

$$Q_{\text{nuevo}}(S_3, \downarrow) = 0.0 + 0.50 \times [-1 + 0.9(0) - 0] = \mathbf{-0.50}$$

Episodio 1 – Paso 4: ¡El gran momento, llegada a la Meta!

OptiBot está en S_6 (justo arriba de la Meta S_9):

1. Elige moverse hacia el **Sur (↓)**.
2. ¡Aterriza en la Meta (S_9)! El entorno le entrega una recompensa masiva: $R_4 = +\mathbf{10.0}$.
3. Como S_9 es un estado terminal que concluye la misión, el valor futuro a partir de la meta es estrictamente cero ($\max_{a'} Q(S_9, a') = 0$).
4. Calculamos el Objetivo de Bellman:

$$\text{Objetivo} = R_4 + \gamma(0) = +\mathbf{10.0}$$

5. Calculamos el Error TD:

$$\delta = \text{Objetivo} - Q(S_6, \downarrow) = +10.0 - 0.0 = +\mathbf{10.0}$$

6. Actualizamos la celda $Q(S_6, \downarrow)$:

$$Q_{\text{nuevo}}(S_6, \downarrow) = 0.0 + 0.50 \times (+10.0) = +\mathbf{5.00}$$

¡Momento revelador!: La celda que conduce directamente a la Meta $(S_6, \downarrow)$ no tiene un valor negativo; ¡posee un enorme $+5.00$ positivo! La Meta ha encendido su primera señal luminosa en la memoria del agente.
Al finalizar este primer episodio, la Tabla Q queda actualizada como se muestra a continuación:

Estado (Casilla)	Norte (↑)	Sur (↓)	Este (→)	Oeste (←)
S_1 (Inicio)	0.00	0.00	$\mathbf{-0.50}$	0.00
S_2 (Pasillo)	0.00	0.00	$\mathbf{-0.50}$	0.00
S_3 (Pasillo)	0.00	$\mathbf{-0.50}$	0.00	0.00
S_4 (Pasillo)	0.00	0.00	0.00	0.00
S_5 (Muro)	—	—	—	—
S_6 (Pasillo)	0.00	$\mathbf{+5.00}$	0.00	0.00
S_7 (Pasillo)	0.00	0.00	0.00	0.00
S_8 (Pasillo)	0.00	0.00	0.00	0.00
S_9 (Meta)	0.00	0.00	0.00	0.00

Nota: En negrita se destacan las cuatro únicas celdas modificadas tras recorrer la trayectoria $S_1 \xrightarrow{\rightarrow} S_2 \xrightarrow{\rightarrow} S_3 \xrightarrow{\downarrow} S_6 \xrightarrow{\downarrow} S_9$. Las demás acciones y casillas aún no han sido exploradas y permanecen en 0.00.

Episodio 2 – Paso 1: Selección basada en la política derivada de Q (S_1)

OptiBot vuelve a colocarse en la casilla de Inicio (S_1) con su Tabla Q actualizada del Episodio 1.
Siguiendo el algoritmo estricto de Q-Learning, el agente debe elegir la acción A usando una política derivada de Q (en este caso, la regla codiciosa $\arg\max_a Q(S_1, a)$). Consultamos la fila de S_1 en la tabla:

$$Q(S_1, \uparrow) = 0.00, \quad Q(S_1, \downarrow) = 0.00, \quad Q(S_1, \rightarrow) = \mathbf{-0.50}, \quad Q(S_1, \leftarrow) = 0.00$$

Decisión crucial y la paradoja de los ceros: El valor más alto en la fila es $\max_a Q(S_1, a) = \mathbf{0.00}$, correspondiente a $\{\uparrow, \downarrow, \leftarrow\}$. La acción Este ($\rightarrow$), penalizada con -0.50 tras el paso inicial del Episodio 1, **no es la opción más alta** y queda descartada por la regla codiciosa.
¿Por qué Norte y Oeste tienen 0.00, pareciendo erróneamente mejores que Este (-0.50)? Porque son acciones aún no exploradas. En Q-Learning, los ceros iniciales representan una ausencia total de experiencia (optimismo inicial). Si OptiBot intentara en este paso moverse al Norte o al Oeste, se saldría de la cuadrícula, sufriendo la penalización terminal catastrófica de $R = -10.0$ (*Game Over*). En una sola actualización de Bellman, dicha acción se desplomaría a:

$$Q_{\text{nuevo}}(S_1, \uparrow) = 0.00 + 0.50 \times [-10.0 + \gamma(0) - 0.00] = \mathbf{-5.00}$$

Por tanto, los ceros en los bordes no son valores ventajosos: son una ilusión transitoria que en cuanto se explora se transforma en números fuertemente negativos (-5.00 y con más visitas hasta -10.00).
Desempatando entre las opciones con valor máximo (0.00) en favor de la dirección transitable que no causa Game Over, OptiBot selecciona la acción: **Sur ($\downarrow$)**.

1. Selecciona la acción con mayor valor transitable: **Sur ($\downarrow$)**.
2. Aterriza en la casilla S_4 y recibe la penalización de paso $R_1 = -1.0$.
3. Consulta la fila de la casilla destino S_4 en su tabla. Como aún no ha sido explorada, todas sus acciones valen 0.00:

$$\max_{a'} Q(S_4, a') = \max(0.00,\ 0.00,\ 0.00,\ 0.00) = \mathbf{0.00}$$

4. Calcula el Objetivo de Bellman:

$$\text{Objetivo} = R_1 + \gamma \max_{a'} Q(S_4, a') = -1.0 + (0.90 \times 0.00) = \mathbf{-1.00}$$

5. Calcula el Error TD:

$$\delta = \text{Objetivo} - Q(S_1, \downarrow) = -1.00 - 0.00 = \mathbf{-1.00}$$

6. Actualiza la celda $Q(S_1, \downarrow)$:

$$Q_{\text{nuevo}}(S_1, \downarrow) = 0.00 + 0.50 \times (-1.00) = \mathbf{-0.50}$$

Episodio 2 – Paso 2: Explorando el pasillo inferior (S_4)

OptiBot se encuentra en S_4:

1. Todas las acciones en S_4 valen 0.00 ($\max_a Q(S_4, a) = 0.00$). Como hacia el Este chocaría con el Muro (S_5), Norte retrocedería y Oeste golpea la pared perimetral, desempata eligiendo la opción máxima hacia la casilla transitable: **Sur (↓)**.
2. Aterriza en la casilla S_7 y recibe la penalización de paso $R_2 = -1.0$.
3. Inspecciona la fila de S_7 en su tabla, donde todas las acciones aún valen 0.00:

$$\max_{a'} Q(S_7, a') = \max(0.00,\ 0.00,\ 0.00,\ 0.00) = \mathbf{0.00}$$

4. Calcula el Objetivo de Bellman:

$$\text{Objetivo} = R_2 + \gamma \max_{a'} Q(S_7, a') = -1.0 + (0.90 \times 0.00) = \mathbf{-1.00}$$

5. Calcula el Error TD:

$$\delta = \text{Objetivo} - Q(S_4, \downarrow) = -1.00 - 0.00 = \mathbf{-1.00}$$

6. Actualiza la celda $Q(S_4, \downarrow)$:

$$Q_{\text{nuevo}}(S_4, \downarrow) = 0.00 + 0.50 \times (-1.00) = \mathbf{-0.50}$$

Episodio 2 – Paso 3: Girando hacia la Meta (S_7)

OptiBot está en la esquina inferior izquierda (S_7):

1. En S_7, todas las acciones valen 0.00 ($\max_a Q(S_7, a) = 0.00$). Desempatando hacia la salida abierta del pasillo, elige **Este (→)**.
2. Aterriza en S_8 recibiendo la penalización de paso $R_3 = -1.0$.
3. Al consultar la fila de la casilla destino S_8, todas sus acciones aún valen 0.00:

$$\max_{a'} Q(S_8, a') = \max(0.00,\ 0.00,\ 0.00,\ 0.00) = \mathbf{0.00}$$

4. Calcula el Objetivo de Bellman:

$$\text{Objetivo} = R_3 + \gamma \max_{a'} Q(S_8, a') = -1.0 + (0.90 \times 0.00) = \mathbf{-1.00}$$

5. Calcula el Error TD:

$$\delta = \text{Objetivo} - Q(S_7, \rightarrow) = -1.00 - 0.00 = \mathbf{-1.00}$$

6. Actualiza la celda $Q(S_7, \rightarrow)$:

$$Q_{\text{nuevo}}(S_7, \rightarrow) = 0.00 + 0.50 \times (-1.00) = \mathbf{-0.50}$$

Episodio 2 – Paso 4: Descubrimiento de la Meta por el Sur (S_8)

OptiBot está en S_8, a la puerta sur de la Meta:

1. Todas las acciones en S_8 valen 0.00 ($\max_a Q(S_8, a) = 0.00$). Elige moverse al **Este (→)**.
2. ¡Aterriza en la Meta (S_9)! Recibe la recompensa masiva $R_4 = +\mathbf{10.0}$.
3. Como S_9 es el estado terminal, el valor futuro es estrictamente cero ($\max_{a'} Q(S_9, a') = 0$).
4. Calcula el Objetivo de Bellman:

$$\text{Objetivo} = R_4 + \gamma(0) = +\mathbf{10.00}$$

5. Calcula el Error TD:

$$\delta = \text{Objetivo} - Q(S_8, \rightarrow) = +10.00 - 0.00 = +\mathbf{10.00}$$

6. Actualiza la celda $Q(S_8, \rightarrow)$:

$$Q_{\text{nuevo}}(S_8, \rightarrow) = 0.00 + 0.50 \times (+10.00) = +\mathbf{5.00}$$

Al finalizar el segundo episodio, la Tabla Q queda actualizada con las dos rutas simétricas descubiertas:

Estado (Casilla)	Norte (↑)	Sur (↓)	Este (→)	Oeste (←)
S_1 (Inicio)	0.00	$\mathbf{-0.50}$	-0.50	0.00
S_2 (Pasillo)	0.00	0.00	-0.50	0.00
S_3 (Pasillo)	0.00	-0.50	0.00	0.00
S_4 (Pasillo)	0.00	$\mathbf{-0.50}$	0.00	0.00
S_5 (Muro)	—	—	—	—
S_6 (Pasillo)	0.00	$+5.00$	0.00	0.00
S_7 (Pasillo)	0.00	0.00	$\mathbf{-0.50}$	0.00
S_8 (Pasillo)	0.00	0.00	$\mathbf{+5.00}$	0.00
S_9 (Meta)	0.00	0.00	0.00	0.00

Interpretación del Episodio 2: El robot descubrió la segunda ruta óptima hacia la meta. Ahora ambos accesos inmediatos a S_9 ($Q(S_6, \downarrow) = +5.00$ por el norte y $Q(S_8, \rightarrow) = +5.00$ por el sur) poseen valores positivos idénticos.

Episodio 3: ¡La propagación de la recompensa hacia atrás!

En el tercer episodio, OptiBot vuelve a situarse en el Inicio (S_1). Ahora, las dos salidas exploradas están empatadas en la cima de las opciones viables ($Q(S_1, \rightarrow) = -0.50$ y $Q(S_1, \downarrow) = -0.50$). Al desempatar hacia el corredor superior (**Este**), observemos lo que ocurre en cada paso:

1. **Paso 1 ($S_1 \rightarrow S_2$):** Se mueve al Este. Con $R_1 = -1.0$ y $\max_{a'} Q(S_2, a') = 0.00$:

$$\delta = -1.00 - (-0.50) = -0.50 \implies Q_{\text{nuevo}}(S_1, \rightarrow) = -0.50 + 0.50(-0.50) = -\mathbf{0.75}$$

2. **Paso 2 ($S_2 \to S_3$):** Avanza al Este. Con $R_2 = -1.0$ y $\max_{a'} Q(S_3, a') = 0.00$:

$$\delta = -1.00 - (-0.50) = -0.50 \implies Q_{\text{nuevo}}(S_2, \to) = -0.50 + 0.50(-0.50) = \mathbf{-0.75}$$

3. **Paso 3 ($S_3 \to S_6$, ¡El salto de valor!):** OptiBot está en S_3 y elige descender al **Sur (↓)**. Aterriza en S_6 recibiendo $R_3 = -1.0$.

   Al consultar la fila del estado destino S_6, ¡ocurre el fenómeno clave!: la casilla hacia el Sur ya tiene el valor positivo descubierto en el Episodio 1:

$$\max_{a'} Q(S_6, a') = \max(0.00,\ \mathbf{+5.00},\ 0.00,\ 0.00) = \mathbf{+5.00}$$

   El Objetivo de Bellman para S_3 se beneficia directamente de esta señal:

$$\text{Objetivo} = R_3 + \gamma \max_{a'} Q(S_6, a') = -1.0 + (0.90 \times 5.00) = -1.0 + 4.50 = \mathbf{+3.50}$$

   El Error TD es intensamente positivo:

$$\delta = \text{Objetivo} - Q(S_3, \downarrow) = +3.50 - (-0.50) = \mathbf{+4.00}$$

   Actualizamos la celda $Q(S_3, \downarrow)$:

$$Q_{\text{nuevo}}(S_3, \downarrow) = -0.50 + 0.50 \times (+4.00) = -0.50 + 2.00 = \mathbf{+1.50}$$

4. **Paso 4 ($S_6 \to S_9$, Consolidación en la Meta):** En S_6, la acción Sur vale $+5.00$ (la más alta). OptiBot avanza al Sur hacia la Meta S_9 recibiendo $R_4 = +10.0$:

$$\text{Objetivo} = +10.0 + 0 = +10.00, \quad \delta = +10.00 - 5.00 = +5.00$$

$$Q_{\text{nuevo}}(S_6, \downarrow) = 5.00 + 0.50 \times (+5.00) = \mathbf{+7.50}$$

Al finalizar el tercer episodio, la Tabla Q refleja con claridad la propagación de la recompensa:

Estado (Casilla)	Norte (↑)	Sur (↓)	Este (→)	Oeste (←)
S_1 (Inicio)	0.00	−0.50	**−0.75**	0.00
S_2 (Pasillo)	0.00	0.00	**−0.75**	0.00
S_3 (Pasillo)	0.00	**+1.50**	0.00	0.00
S_4 (Pasillo)	0.00	−0.50	0.00	0.00
S_5 (Muro)	—	—	—	—
S_6 (Pasillo)	0.00	**+7.50**	0.00	0.00
S_7 (Pasillo)	0.00	0.00	−0.50	0.00
S_8 (Pasillo)	0.00	0.00	+5.00	0.00
S_9 (Meta)	0.00	0.00	0.00	0.00

Nota explicativa: Se observa con precisión matemática cómo la recompensa retrocede casilla por casilla: la antesala de la meta $(S_6, \downarrow)$ sube a $+7.50$ y la casilla previa $(S_3, \downarrow)$ salta por primera vez a un valor fuertemente positivo $(+1.50)$.

La intuición de las migajas de pan y el optimismo ante la incertidumbre: Este proceso ilustra dos pilares esenciales de Q-Learning:

1. **Exploración por optimismo inicial:** En el Episodio 2, el robot no repitió a ciegas la ruta norte porque su penalización la redujo a -0.50, mientras que las opciones no visitadas ofrecían 0.00. Esto empujó al agente a descubrir la ruta alternativa sur.
2. **Propagación hacia atrás de la recompensa:** La señal masiva de la Meta $(+10)$ retrocede un paso en cada recorrido completo. Primero iluminó la casilla vecina S_6 con $+5.00$, y en la siguiente visita iluminó la casilla previa S_3 con $+1.50$. Conforme avanzan los episodios, esta ola positiva llega a la casilla inicial (S_1), consolidando la política óptima.

5.7 Evolución a Largo Plazo: Cómo los Ceros se Transforman en Valores Negativos

Si las acciones transitables acumulan penalizaciones iniciales $(-0.50, -0.75)$, ¿por qué el robot no prefiere indefinidamente las acciones que aún valen 0.00?

La respuesta es que esos ceros reflejan únicamente falta de experiencia. Conforme el agente explora en episodios sucesivos $(\varepsilon \approx 1.0)$, descubre rápidamente el costo real de las malas decisiones:

1. **Salirse del laberinto (Game Over):** Al intentar cruzar los bordes perimetrales de la cuadrícula, el agente recibe una penalización inmediata de $R = -10.0$ y el episodio concluye (*Game Over*). Al ser un estado terminal sin futuro $(\max_{a'} Q = 0)$, la regla de actualización reduce drásticamente su valor:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[-10.0 + \gamma(0) - Q(s,a)\Big]$$

   En la primera visita con $\alpha = 0.50$, la celda cae de 0.00 a $\mathbf{-5.00}$, y en pocas iteraciones converge a $\mathbf{-10.00}$. Los ceros engañosos de los bordes se transforman así en valores fuertemente negativos.

2. **Chocar contra el muro central (S_5):** Intentar ingresar en la estantería bloqueada hace rebotar al robot en su misma casilla, sumando penalizaciones de paso $(R = -1.0)$ sin acercarlo a la meta.

5.7.1 El auge de los pasillos interiores: Descubrimiento de dos rutas óptimas

Mientras que las salidas prohibidas caen al fondo de la escala (≈ -10.00), los pasillos transitables experimentan el efecto contrario: aunque cada movimiento descuenta -1.00, están conectados a la enorme recompensa de $+10.0$ de la Meta (S_9).

A medida que el robot completa cientos de episodios, la recompensa positiva retrocede paso a paso hasta alcanzar el punto de partida (S_1). Al converger el entrenamiento, la Tabla Q queda completamente estructurada:

Estado (Casilla)	Norte (↑)	Sur (↓)	Este (→)	Oeste (←)
S_1 (Inicio)	-10.00	$\mathbf{+4.58}$	$\mathbf{+4.58}$	-10.00
S_2 (Pasillo)	-10.00	$+4.58$	$\mathbf{+6.20}$	$+3.12$
S_3 (Pasillo)	-10.00	$\mathbf{+8.00}$	-10.00	$+4.58$
S_4 (Pasillo)	$+3.12$	$\mathbf{+6.20}$	$+4.58$	-10.00
S_5 (Muro)	—	—	—	—
S_6 (Pasillo)	$+6.20$	$\mathbf{+10.00}$	-10.00	$+8.00$
S_7 (Pasillo)	$+4.58$	-10.00	$\mathbf{+8.00}$	-10.00
S_8 (Pasillo)	$+8.00$	-10.00	$\mathbf{+10.00}$	$+6.20$
S_9 (Meta)	0.00	0.00	0.00	0.00

Interpretación de la convergencia final:

- Las acciones que intentan abandonar el laberinto se han fijado en $\mathbf{-10.00}$, por lo que una política codiciosa jamás las seleccionará.
- En el inicio (S_1), el agente descubre que existen **dos caminos óptimos y simétricos** de 4 pasos con el mismo valor óptimo ($Q^* = +4.58$):
  - **Camino Superior:** $S_1 \xrightarrow{\text{Este}} S_2 \xrightarrow{\text{Este}} S_3 \xrightarrow{\text{Sur}} S_6 \xrightarrow{\text{Sur}} S_9$.
  - **Camino Inferior:** $S_1 \xrightarrow{\text{Sur}} S_4 \xrightarrow{\text{Sur}} S_7 \xrightarrow{\text{Este}} S_8 \xrightarrow{\text{Este}} S_9$.

5.8 Visualización Gráfica: La Política Aprendida en la Cuadrícula

Una vez que el algoritmo converge, ¿cómo se interpreta la Tabla Q en el espacio físico del almacén?

Para cada casilla s, simplemente dibujamos una flecha apuntando hacia la acción con el valor Q más elevado. La Figura 5.2 muestra el resultado:

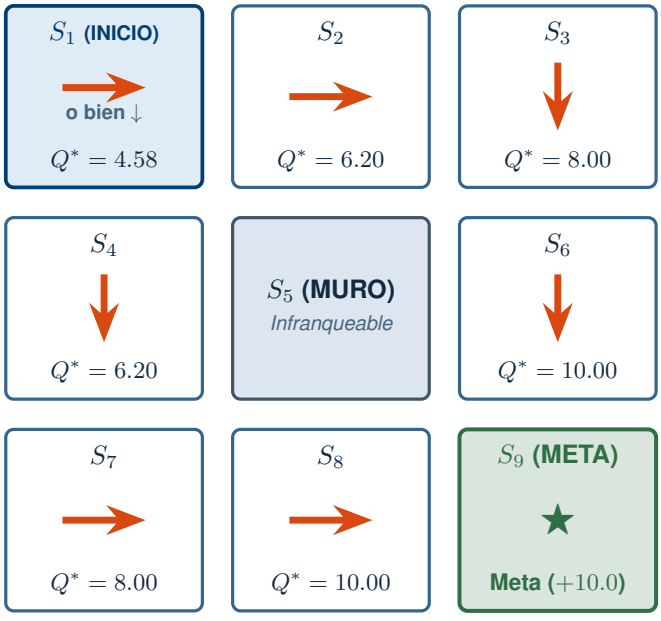


Figura 5.2: Política de navegación óptima aprendida por Q-Learning sobre la cuadrícula. Cada flecha en color coral indica la acción óptima $\pi^*(s) = \arg\max_a Q^*(s,a)$ en dicho estado, y el valor inferior muestra $Q^*(s, \pi^*(s))$. Obsérvese cómo las flechas guían inequívocamente a OptiBot bordeando el obstáculo central (S_5) a través de dos rutas mínimas de 4 pasos hacia la Meta (S_9).

5.9 Implementación en Python: El Laberinto de OptiBot

Para experimentar de forma interactiva con el entorno de simulación y el algoritmo tabular de Q-Learning, el código completo se encuentra disponible en el cuaderno Jupyter:

`notebooks/cap04_rl_tradicional.ipynb`

En este cuaderno interactivo se implementa la clase modular `LaberintoOptiBot` con la cuadrícula de 3×3, penalizaciones de frontera (-10.0 con *Game Over*) y recompensa por meta ($+10.0$). Asimismo, se programa el agente tabular `AgenteQLearning` con decaimiento de exploración ε, visualizando en tiempo real la convergencia de la tabla Q, las curvas de recompensa acumulada y la política direccional aprendida que conduce a la meta bordeando el obstáculo central por ambas rutas simétricas.

5.10 La Gran Limitación: La Maldición de la Dimensionalidad

La cuadrícula de 3×3 tiene apenas 9 estados y 4 acciones, por lo que la Tabla Q cabe cómodamente en cualquier hoja de cálculo con 36 celdas.

Sin embargo, ¿qué ocurriría si OptiBot saliera de la cuadrícula discreta de juguete y operara en un almacén real con sensores continuos?

1. **Sensores continuos:** Un sensor mide distancias con infinitos decimales ($d \in [0.05, 15.0]$ m), su velocímetro mide velocidades reales ($v \in [0.0, 2.5]$ m/s) y su batería decrece de forma continua ($b \in [0.0, 100.0]\,\%$).
2. **Espacio infinito de estados:** El número de combinaciones posibles en un entorno continuo es literalmente **infinito** ($|\mathcal{S}| = \infty$).
3. **El límite de las tablas:** No existe memoria RAM en el planeta que pueda almacenar una tabla de infinitas filas. Más aún, un robot jamás visitaría cada milímetro del espacio las veces requeridas para que cada celda aprenda.

Para sistemas reales, necesitamos sustituir la tabla rígida por un **aproximador continuo inteligente** capaz de generalizar: las **Redes Neuronales Profundas**.

El puente hacia el siguiente capítulo: En el Capítulo 6, daremos el gran salto conceptual hacia el **Aprendizaje por Refuerzo Profundo (Deep Q-Learning o DQN)**. Descubriremos cómo sustituir la Tabla Q por una red neuronal, por qué esta combinación puede ser inestable si no se controla adecuadamente (la *Tríada Mortal*), y cómo la arquitectura DQN de DeepMind logró estabilizar el aprendizaje mediante el buffer de repetición de experiencias y las redes objetivo congeladas.

6

Aprendizaje por Refuerzo Profundo: Deep Q-Learning (DQN)

6.1 De Tablas Discretas a Aproximadores de Funciones Continuos

En el Capítulo 4 (Aprendizaje por Refuerzo Tradicional), descubrimos la elegancia del algoritmo Q-Learning tabular: un agente aprende a desenvolverse en un mundo cuadriculado anotando números pacientemente en una libreta (la Tabla Q). Sin embargo, este método choca frontalmente contra un muro insalvable cuando intenta salir al mundo real: la temida **maldición de la dimensionalidad**.

6.1.1 ¿Por qué las Tablas Q no Funcionan en el Mundo Real?

Imaginemos que queremos que nuestro robot OptiBot aprenda a navegar en un almacén físico de 3×3 metros. En la vida real, el suelo no está dividido en 9 baldosas gigantescas; el robot se mueve en un espacio de **coordenadas métricas continuas**:

- Si dividiéramos el almacén en casillas de un centímetro (1 cm $\times$ 1 cm), tendríamos $300 \times 300 = 90{,}000$ casillas. Para 4 acciones cardinales, la tabla requeriría $360{,}000$ celdas.
- Si quisiéramos precisión milimétrica (1 mm), la tabla se dispararía a ¡36 millones de celdas!
- Y si además el robot mide su velocidad, aceleración o ángulo de giro, el número de combinaciones posibles se vuelve astronómico. La memoria de cualquier ordenador colapsaría al instante.

Pero el problema no es solo de memoria; es de **sentido común**:

> *Una tabla Q es incapaz de asociar ideas. Si OptiBot descubre en su tabla que la posición $(0.50, 2.50)$ m es segura y que debe avanzar al Este, pero un sensor reporta $(0.51, 2.49)$ m (una diferencia de un solo centímetro), la tabla Q trata esa coordenada como un universo totalmente desconocido. ¡El robot tendría que cometer los mismos errores una y otra vez para aprender cada milímetro por separado!*

Los seres humanos no memorizamos cada milímetro de nuestra cocina para no chocarnos con la mesa. Poseemos una facultad cognitiva extraordinaria: la **generalización**. Si sabemos esquivar un obstáculo en un rincón, sabemos extrapolar esa intuición a cualquier posición similar. Para dotar a una máquina de esa misma habilidad de generalización, jubilamos la tabla Q y contratamos a una **red neuronal artificial**.

6.1.2 La Red Neuronal como Calculadora de Perillas Inteligentes

Una red neuronal suele describirse como un misterio impenetrable, pero en aprendizaje por refuerzo podemos visualizarla con total sencillez: es una **calculadora interactiva con miles de perillas ajustables** (los pesos numéricos $\boldsymbol{\theta}_{\text{policy}}$):

1. **Entrada (los ojos del robot):** En lugar de buscar una casilla en una tabla, le inyectamos directamente los números de los sensores continuos de OptiBot: su posición métrica (x, y) y la distancia que le falta para llegar a la meta $(\Delta x_{\text{meta}}, \Delta y_{\text{meta}})$.

2. **Cuerpo interno (las perillas ajustables $\boldsymbol{\theta}_{\text{policy}}$):** La señal viaja a través de varias capas de neuronas interconectadas. Cada conexión cuenta con una "perilla" (peso) que amplifica o atenúa la señal eléctrica.

3. **Salida (el menú de recomendaciones):** En el extremo final, la red produce simultáneamente cuatro números decimales: uno para cada acción posible:

$$\big[Q(\mathbf{s}, \text{Norte}),\ Q(\mathbf{s}, \text{Sur}),\ Q(\mathbf{s}, \text{Este}),\ Q(\mathbf{s}, \text{Oeste})\big]$$

   Cada valor representa la expectativa de recompensa acumulada futura. OptiBot solo tiene que consultar cuál de las cuatro salidas es la más alta para decidir qué dirección tomar.

4. **¿Qué significa aprender?:** Aprender ya no consiste en rellenar casillas de una hoja de cálculo, sino en **girar ligeramente esas perillas internas ($\boldsymbol{\theta}_{\text{policy}}$)** cada vez que el robot experimenta una sorpresa en el almacén, de modo que sus predicciones futuras sean cada vez más acertadas:

$$Q(\mathbf{s}, a; \boldsymbol{\theta}_{\text{policy}}) \approx Q^*(\mathbf{s}, a) \tag{6.1}$$

En este capítulo, OptiBot regresa al almacén de 3×3 metros del Capítulo 4, pero ahora despojado de baldosas discretas y enfrentándose a un **entorno físico continuo**:

- **Dimensiones métricas:** El suelo es una sala cuadrada de 3.0 m $\times$ 3.0 m. Las coordenadas del robot son números decimales continuos: $0.0 \leq x \leq 3.0$ y $0.0 \leq y \leq 3.0$.
- **Muelle de Carga (S_1, Inicio):** OptiBot parte en la esquina superior izquierda, cerca de las coordenadas iniciales $(x_0, y_0) = (0.55 \text{ m}, 2.35 \text{ m})$.
- **Estantería Central Infranqueable (S_5):** Un obstáculo cúbico macizo en el centro exacto de la sala ($1.0 \leq x \leq 2.0$ y $1.0 \leq y \leq 2.0$). Si OptiBot intenta ingresar, colisiona físicamente y rebota.
- **Bahía de Entrega (S_9, Meta):** La zona objetivo en la esquina inferior derecha ($2.0 \leq x \leq 3.0$ e $0.0 \leq y \leq 1.0$, con centro de atracción en $x = 2.50$ m, $y = 0.75$ m). Al alcanzarla, el robot completa la misión con una gran bonificación de $+10.0$.
- **Paredes perimetrales:** Si el robot sale de los límites ($x < 0$, $x > 3$, $y < 0$ o $y > 3$), choca contra la pared exterior sufriendo una fuerte penalización de $R = -10.0$.

En cada paso t, la telemetría de OptiBot extrae un vector de estado continuo de cuatro números $\mathbf{s} \in \mathbb{R}^4$:

1. $x \in [0.0, 3.0]$ m: Coordenada horizontal actual en el almacén.

2. $y \in [0.0, 3.0]$ m: Coordenada vertical actual en el almacén.
3. $\Delta x_{\text{meta}} = x_{\text{meta}} - x$: Desplazamiento horizontal relativo hacia la Bahía de Entrega.
4. $\Delta y_{\text{meta}} = y_{\text{meta}} - y$: Desplazamiento vertical relativo hacia la Bahía de Entrega.

Este vector continuo ingresa directamente en la red neuronal, la cual emite los valores $Q(\mathbf{s}, a)$ para las cuatro maniobras cardinales:

$$\mathcal{A} = \{\text{Norte } (\uparrow), \text{ Sur } (\downarrow), \text{ Este } (\rightarrow), \text{ Oeste } (\leftarrow)\}$$

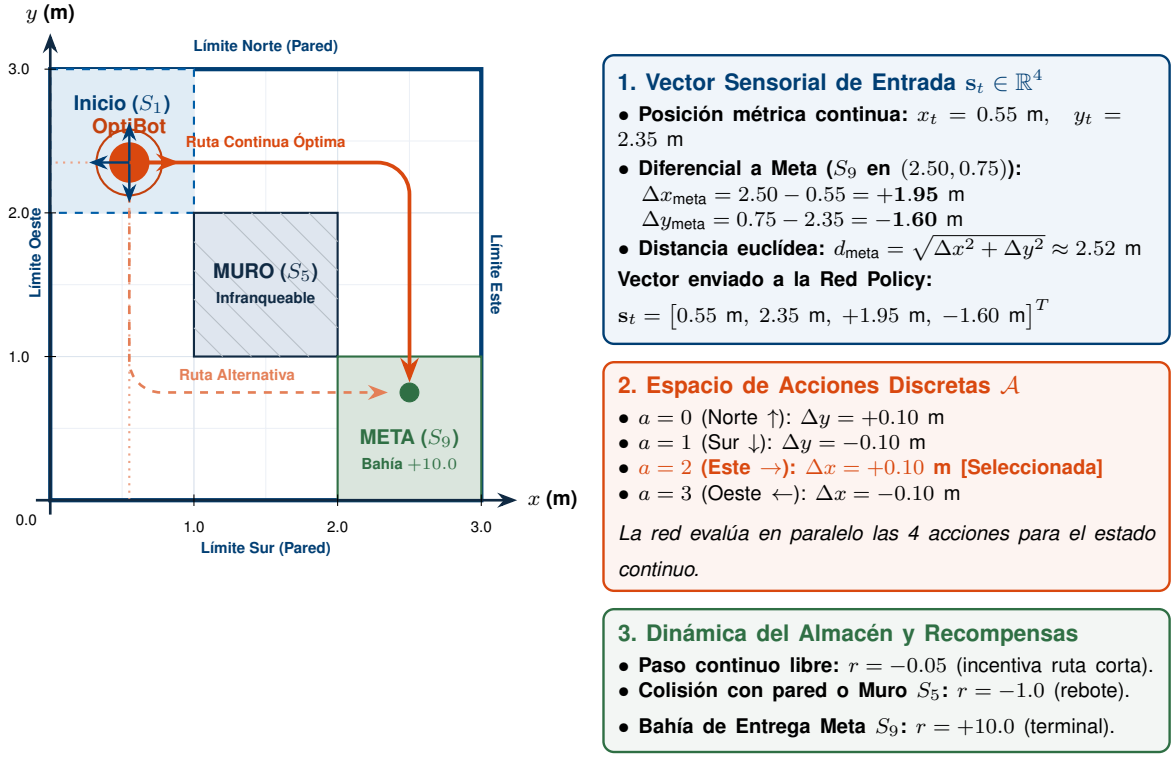


Figura 6.1: El entorno continuo del almacén de OptiBot (3 m $\times$ 3 m) con arquitectura de telemetría sensorial. A la izquierda: mapa espacial continuo con las trayectorias óptimas aprendidas para rodear el muro central (S_5). A la derecha: descomposición del vector sensorial de 4 dimensiones $\mathbf{s}_t = [x, y, \Delta x_{\text{meta}}, \Delta y_{\text{meta}}]^T$, el menú de acciones cardinales y las reglas de recompensa que alimentan a la red neuronal.

Como ilustra la Figura 6.1, OptiBot ya no salta torpemente entre celdas; se desliza fluidamente en un mundo físico real. Para procesar este vector sensorial $\mathbf{s}$ en tiempo real, OptiBot recurre a una arquitectura que combina dos redes neuronales idénticas en estructura, como se aprecia en la Figura 6.2.

6.2 El Algoritmo Deep Q-Learning: Las Dos Redes y el Flujo Maestro

Sustituir una tabla por una red neuronal parece una idea obvia, pero durante dos décadas fue considerado una misión casi imposible en inteligencia artificial. Cuando los investigadores intentaban conectar una red neuronal a la ecuación de Bellman en los años 90, el entrenamiento

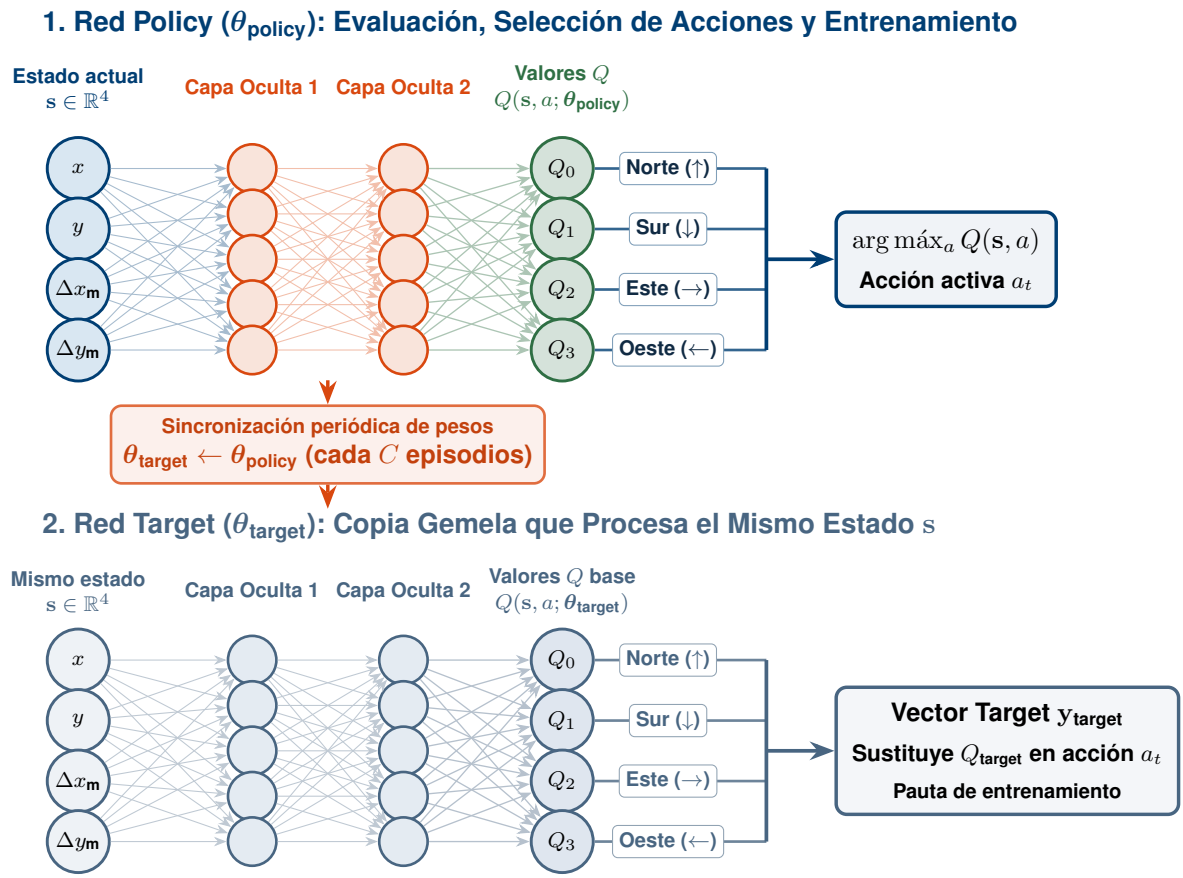


Figura 6.2: Arquitectura dual de Deep Q-Network (DQN) para la navegación continua de OptiBot. Arriba: la **Red Policy** ($\boldsymbol{\theta}_{\text{policy}}$), donde entrena el agente y predice los valores de acción en tiempo real a partir del estado capturado $\mathbf{s} = [x, y, \Delta x_{\text{m}}, \Delta y_{\text{m}}]$. Abajo: la **Red Target** ($\boldsymbol{\theta}_{\text{target}}$), copia gemela que procesa el **mismo estado actual** $\mathbf{s}$ para conformar la pauta del vector target $\mathbf{y}_{\text{target}}$, sustituyendo únicamente la acción ejecutada por el blanco de Bellman. En el centro, la sincronización periódica de parámetros transfiere los pesos aprendidos de la red policy a la red target.

fallaba estrepitosamente: la red empezaba a arrojar predicciones absurdas, los valores oscilaban sin control y el sistema divergía al infinito.

¿Por qué ocurría este desastre? Porque el aprendizaje por refuerzo rompe las dos reglas sagradas del aprendizaje automático supervisado clásico:

- **Problema 1: La trampa del tiempo y el olvido catastrófico (Correlación Temporal).** En visión artificial, le mostramos a la red millones de fotos barajadas al azar (un perro, luego un coche, luego un gato). Pero un robot que camina por un pasillo experimenta secuencias fuertemente dependientes en el tiempo: "paso adelante, paso adelante, paso adelante". Si entrenamos a la red solo con lo que acaba de pasar, sufrirá de **olvido catastrófico**: aprenderá a avanzar en línea recta, pero olvidará por completo cómo girar en una esquina.
- **Problema 2: Disparar a una diana sobre patines (El Blanco Móvil).** En aprendizaje tradicional, la respuesta correcta de un examen es fija. Pero en Q-Learning, ¡la red calcula su propia respuesta correcta usando la ecuación de Bellman! Si usamos una única red para predecir el valor actual y también para calcular el objetivo futuro, cada vez que giremos las perillas para acercarnos a la diana, ¡la diana misma se desplazará de sitio! Es como intentar clavar una flecha en un blanco que huye sobre patines atado a tus propios pies.

En 2013 y 2015, los científicos de **DeepMind** (Mnih et al.) revolucionaron la computación al resolver de raíz estos dos dilemas mediante el algoritmo **Deep Q-Networks (DQN)**, introduciendo dos innovaciones maestras: el **Experience Replay** (el álbum de recuerdos) y la **Arquitectura Dual con Red Target Congelada**.

A continuación, descompondremos el funcionamiento de DQN a través de un viaje estructurado en 5 fases lógicas y 10 pasos operativos.

6.2.1 Fase 1: El Nacimiento de las Dos Redes Gemelas

El algoritmo arranca construyendo dos copias arquitectónicas idénticas de la misma red neuronal:

- **Paso 1: Creación de la Red Policy (θ_{policy}).** Se inicializan sus perillas (pesos y sesgos) con valores aleatorios diminutos. En este instante inicial, OptiBot no sabe nada del almacén: no distingue un pasillo despejado de un muro impenetrable. Esta será la red "estudiante": la que interactuará con el entorno, tomará decisiones y se entrenará paso a paso.
- **Paso 2: Clonación y congelación de la Red Target (θ_{target}).** Se realiza una copia idéntica en la memoria del ordenador:

$$\boldsymbol{\theta}_{\text{target}} \leftarrow \boldsymbol{\theta}_{\text{policy}} \tag{6.2}$$

  Ambas redes tienen exactamente los mismos cables y perillas. Sin embargo, la Red Target se coloca en un "congelador" temporal: sus perillas quedan bloqueadas y no se actualizarán en cada paso. Ella será la "examinadora" imparcial que sostendrá la diana de Bellman completamente inmóvil.

6.2.2 Fase 2: Vivir la Aventura y Guardar Fotos en el Álbum

Con las dos mentes preparadas, OptiBot sale a explorar el almacén:

- **Paso 3: Percepción sensorial y acción.** El robot lee su posición métrica continua:

$$\mathbf{s}_t = [x_t,\, y_t,\, \Delta x_{\text{meta}},\, \Delta y_{\text{meta}}]^T \tag{6.3}$$

  Para decidir qué hacer, utiliza la conocida regla ε-greedy: la mayor parte del tiempo $(1-\varepsilon)$ consulta a su Red Policy y escoge la acción con mayor valor Q; pero a veces (ε) explora una dirección al azar. El robot ejecuta el movimiento elegido a_t, el entorno físico responde, recibe una recompensa r_{t+1} y aterriza en una nueva coordenada métrica $\mathbf{s}_{t+1}$.
- **Paso 3a: Guardar la experiencia en el Álbum (*Experience Replay*, $\mathcal{D}$).** En lugar de entrenar de inmediato con lo que acaba de vivir (lo que causaría olvido catastrófico), OptiBot empaqueta su vivencia en una tupla indivisible de 5 elementos:

$$e_t = \left(\mathbf{s}_t,\, a_t,\, r_{t+1},\, \mathbf{s}_{t+1},\, \text{terminal}_{t+1}\right) \tag{6.4}$$

  y la guarda en una memoria de cola circular (FIFO) con capacidad para miles de recuerdos (por ejemplo, $50{,}000$ pasos). Si el álbum se llena, cada recuerdo nuevo expulsa pacíficamente al más antiguo.
- **Paso 3b: Barajar y extraer un lote aleatorio (*Mini-Batch Sampling*).** Para entrenar, el agente no mira la última foto; extrae una muestra aleatoria (un puñado de 32 o 64 recuerdos) dispersos en el tiempo y en el espacio:

$$\mathcal{B} = \{e_1, e_2, \ldots, e_B\} \sim \mathcal{D} \tag{6.5}$$

  ¡Al mezclar recuerdos de esquinas, pasillos y choques ocurridos en momentos distintos, la correlación temporal se destruye por completo!

6.2.3 Fase 3: La Evaluación de Bellman y la Diana Fija

Ahora que tenemos un lote de recuerdos sobre la mesa, examinemos cómo se evalúa una experiencia concreta $(\mathbf{s}, a, r, \mathbf{s}', \text{term})$:

- **Paso 4: La Red Policy evalúa el estado inicial $\mathbf{s}$.** Le mostramos las coordenadas $\mathbf{s}$ a la Red Policy. Esta enciende sus perillas y calcula cuatro números de salida:

$$\big[Q(\mathbf{s}, \text{Norte}),\ Q(\mathbf{s}, \text{Sur}),\ Q(\mathbf{s}, \text{Este}),\ Q(\mathbf{s}, \text{Oeste})\big]$$

  Anotamos la predicción que la red hizo para la acción a que el robot ejecutó realmente: este valor es $Q(\mathbf{s}, a; \boldsymbol{\theta}_{\text{policy}})$.

- **Paso 5: La Red Target evalúa ese mismo estado inicial $\mathbf{s}$.** Introducimos las mismas coordenadas $\mathbf{s}$ en la Red Target congelada para obtener sus valores de referencia base:

$$Q(\mathbf{s}, \cdot; \boldsymbol{\theta}_{\text{target}})$$

- **Paso 6: Cálculo del valor objetivo de Bellman (Q_{target}).** ¿Cuál debió haber sido la nota perfecta para esa acción? Para saberlo, consultamos el futuro: introducimos el **siguiente estado alcanzado** $\mathbf{s}'$ en la **Red Target congelada**. Aplicamos la ecuación de Bellman:

  - Si el paso terminó el episodio (terminal $=$ True): el juego concluyó y no hay futuro:

$$Q_{\text{target}} = r \tag{6.6}$$

  - Si el robot continúa navegando (terminal $=$ False): sumamos la recompensa inmediata y el valor de la mejor opción futura descontada:

$$Q_{\text{target}} = r + \gamma \max_{a' \in \mathcal{A}} Q(\mathbf{s}', a'; \boldsymbol{\theta}_{\text{target}}) \tag{6.7}$$

  Aspecto clave: Como este valor Q_{target} se calcula con la Red Target (cuyas perillas están congeladas), **este número es una roca sólida**. La diana no se mueve mientras disparamos.

6.2.4 Fase 4: El Truco Maestro del Vector Target y el Gradiente

Aquí surge la pregunta que desconcierta a todo principiante: *«La Red Policy produce cuatro números a la salida, pero nosotros solo conocemos el resultado real de la única acción que OptiBot ejecutó. ¿Cómo entrenamos a la red sin arruinar las otras tres salidas?»*

- **Paso 7: El ingenioso artificio del Vector Target ($\mathbf{y}_{\text{target}}$).** Tomamos las cuatro predicciones que emitió la Red Target en el Paso 5 y **sustituimos únicamente la casilla de la acción ejecutada por el nuevo valor de Bellman Q_{target}**:

$$\mathbf{y}_{\text{target}} = \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \end{bmatrix}, \quad \text{donde } y_k = \begin{cases} Q_{\text{target}} & \text{si } k = a \text{ (la acción que el robot probó)} \\ Q(\mathbf{s}, k; \boldsymbol{\theta}_{\text{target}}) & \text{si } k \neq a \text{ (las 3 acciones no ejecutadas)} \end{cases} \tag{6.8}$$

  ¿Por qué esto es pura genialidad?: Porque al calcular el error para las otras tres acciones que no probamos, restamos lo mismo que la red predijo:

$$\text{Error en acción no probada } k = (y_k - Q_k) = (Q_k - Q_k) = 0$$

  ¡El error en las otras tres acciones es exactamente cero! Sus perillas internas no sufren distorsiones indebidas; el 100 % de la corrección se concentra con precisión de cirujano en la acción que OptiBot puso a prueba.

- **Paso 8: Entrenamiento mediante Descenso de Gradiente (MSE).** Comparamos la predicción de la Red Policy con el vector target usando el Error Cuadrático Medio:

$$\mathcal{L}(\boldsymbol{\theta}_{\text{policy}}) = \left(Q_{\text{target}} - Q(\mathbf{s}, a; \boldsymbol{\theta}_{\text{policy}})\right)^2 \tag{6.9}$$

  Mediante retropropagación (*backpropagation*), el optimizador gira suavemente las perillas $\boldsymbol{\theta}_{\text{policy}}$ de la Red Policy para que su estimación se acerque al blanco de Bellman. La Red Target ni se entera: sus perillas no se tocan (lo que en matemáticas se denomina *semi-gradiente*).

6.2.5 Fase 5: El Ciclo Continuo y la Sincronización Periódica

- **Paso 9: Repetición del bucle interactivo.** OptiBot sigue recorriendo el almacén paso a paso, llenando su álbum de vivencias y entrenando su Red Policy con cada lote aleatorio.
- **Paso 10: Sincronización periódica de las perillas.** La Red Target no puede quedarse congelada para siempre, pues el robot se estancaría intentando complacer a un juez anticuado. Por ello, tras un intervalo programado (por ejemplo, cada $C = 4$ pasos de entrenamiento), descongelamos momentáneamente la Red Target y le copiamos los pesos actualizados de la Red Policy:

$$\boldsymbol{\theta}_{\text{target}} \leftarrow \boldsymbol{\theta}_{\text{policy}} \tag{6.10}$$

  Tras este relevo, la diana da un salto hacia adelante reflejando la nueva sabiduría de OptiBot, pero vuelve a congelarse de inmediato durante otros 4 pasos para garantizar una estabilidad matemática inquebrantable.

6.3 Cómo Aprende la Red: Función de Pérdida y Descenso de Gradiente

Con el vector objetivo establecido gracias a la Red Target, el entrenamiento de la Red Policy se convierte en un proceso de calibración continua: comparar lo que la red predijo contra la evidencia real recolectada y ajustar sus perillas internas para reducir el margen de error.

6.3.1 La Función de Pérdida: Midiendo la Sorpresa del Robot

Para que la red aprenda, primero necesitamos una brújula matemática que cuantifique qué tan equivocadas estuvieron sus predicciones respecto a la meta fijada por Bellman. A esta medida de discrepancia la llamamos **función de pérdida** (o costo), y se basa en el principio del **error cuadrático**:

- **Calcular la brecha:** Tomamos la nota que la Red Policy le dio a la acción antes de ejecutarla y la comparamos con el valor objetivo que calculó la Red Target tras observar el resultado real.
- **Elevar al cuadrado:** Multiplicamos la diferencia por sí misma. Esto cumple dos propósitos fundamentales sin necesidad de fórmulas complicadas:

  1. Hace que cualquier error sea siempre positivo (tanto si el robot fue demasiado optimista como si fue demasiado pesimista, equivocarse siempre cuenta como error).
  2. Castiga con mucha mayor severidad las equivocaciones garrafales que los pequeños desajustes milimétricos, obligando a la red a corregir primero sus fallos más peligrosos.

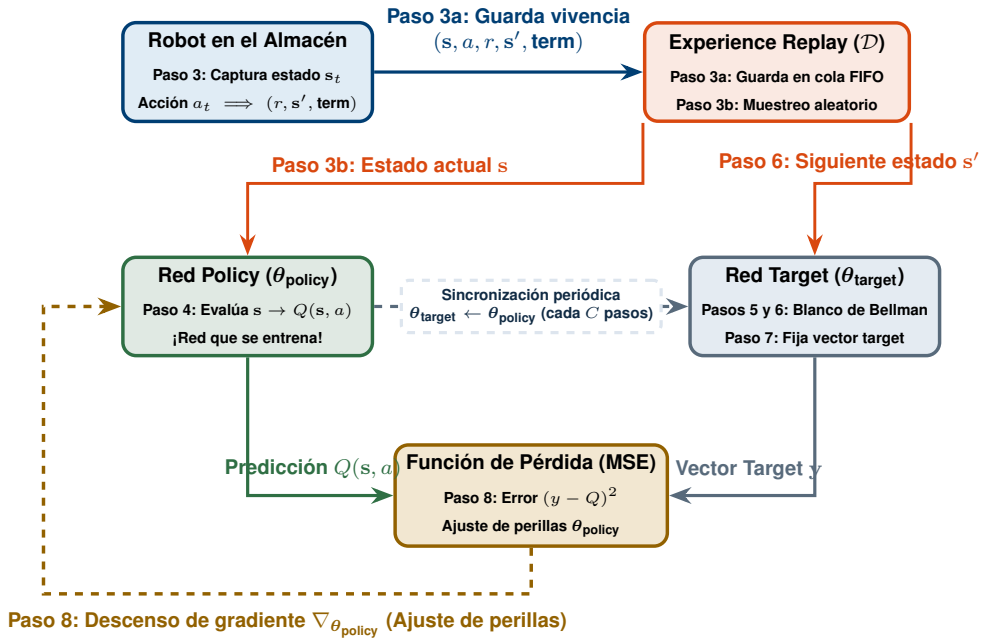


Figura 6.3: Ciclo interactivo del algoritmo Deep Q-Learning (DQN). El robot interactúa con el almacén continuo y empaqueta sus transiciones en el búfer de repetición de experiencias. Para entrenar de forma descorrelacionada, se extrae un lote aleatorio: la **Red Policy** evalúa el estado actual s para obtener su predicción, mientras que la **Red Target congelada** evalúa el siguiente estado s' para computar el blanco inmutable de Bellman y conformar el vector target sustituyendo la acción ejecutada. La discrepancia cuadrática entrena exclusivamente a la Red Policy. Periódicamente (cada C pasos), las perillas aprendidas se transfieren a la Red Target.

- **Promediar sobre el lote:** En lugar de reaccionar de forma impulsiva a un solo recuerdo, calculamos el error promedio sobre las decenas de vivencias barajadas del lote. De esta manera, el aprendizaje es suave, estable y representativo de todo el almacén.

6.3.2 ¿Por qué la sustitución aísla el error en la acción tomada?

Aquí reside uno de los conceptos más ingeniosos de DQN: la red tiene cuatro salidas simultáneas (Norte, Sur, Este y Oeste), pero en cada instante OptiBot solo puede moverse en una dirección. Si el robot avanzó hacia el Este, ¿cómo podemos evaluar a la red sin distorsionar lo que opina sobre las otras tres rutas que no probó?

- **La analogía del restaurante:** Si vas a cenar y pides pasta, al terminar la velada puedes opinar con certeza sobre el sabor de la pasta. Pero sería un disparate cambiar de opinión sobre el pescado o la sopa, porque simplemente no los probaste.
- **El truco del camuflaje:** En la plantilla del vector objetivo, dejamos para las acciones no ejecutadas exactamente el mismo número que la red ya había predicho. Al comparar la predicción contra el objetivo, la diferencia en esas acciones no probadas es exactamente cero.
- **Enfoque quirúrgico:** Como el error en las otras tres direcciones es nulo, el algoritmo no altera las perillas asociadas a ellas. Toda la fuerza de corrección se concentra exclusivamente en la acción que el robot experimentó en carne propia.

Paso	Acción del Algoritmo	Componente Activo	Significado en OptiBot
1	Creación con pesos aleatorios	Red Policy ($\boldsymbol{\theta}_{\text{policy}}$)	Inicializa perillas al azar sin conocer el almacén.
2	Clonación inicial idéntica	Red Target ($\boldsymbol{\theta}_{\text{target}}$)	Copia exacta $\boldsymbol{\theta}_{\text{target}} \leftarrow \boldsymbol{\theta}_{\text{policy}}$ congelada en memoria.
3	Medición sensorial y acción	Sensores continuos	Mide posición continua $[x, y, \Delta x, \Delta y]^T$ y se mueve.
3a	Inserción en Experience Replay	Cola FIFO ($\mathcal{D}$)	Guarda la vivencia $(\mathbf{s}, a, r, \mathbf{s}', \text{term})$ en el álbum.
3b	Muestreo aleatorio de lote	Muestreador de memoria	Extrae recuerdos al azar para romper la correlación temporal.
4	Propagación hacia adelante	Red Policy	Predice las 4 notas cardinales $(\uparrow, \downarrow, \rightarrow, \leftarrow)$ para $\mathbf{s}$.
5	Propagación del mismo estado	Red Target	Emite la plantilla base de predicciones para $\mathbf{s}$.
6	Cómputo del blanco de Bellman	Red Target congelada	Calcula la meta fija: $Q = r + \gamma \max Q(\mathbf{s}')$ (o r si es terminal).
7	Sustitución en Vector Target	Vector $\mathbf{y}_{\text{target}}$	Reemplaza únicamente el índice de la acción a por Q_{target}.
8	Entrenamiento por gradiente	Optimizador (MSE)	Gira las perillas de la Red Policy hacia la diana fija.
9	Repetición del ciclo interactivo	Bucle de navegación	Vuelve al Paso 3 para seguir explorando y aprendiendo.
10	Sincronización periódica	Copia de parámetros	Actualiza la Red Target $\boldsymbol{\theta}_{\text{target}} \leftarrow \boldsymbol{\theta}_{\text{policy}}$ cada C pasos.

Cuadro 6.1: Resumen sinóptico de los 10 pasos fundamentales de Deep Q-Learning (DQN).

6.3.3 El Enfoque del Descenso de Gradiente y el Semi-Gradiente

Una vez que conocemos la magnitud del error, ¿cómo sabe el robot hacia dónde girar sus miles de perillas internas?

- **Descenso de gradiente (caminar en la niebla):** Imaginemos a una persona bajando una montaña empinada en medio de una niebla espesa que no le permite ver la cima ni el valle. Para descender hacia el punto más bajo (el menor error posible), palpa la inclinación del terreno con los pies y da un paso corto en la dirección de mayor descenso. Eso es exactamente el descenso de gradiente: la red evalúa hacia dónde se inclina el error y gira ligeramente cada perilla interna en la dirección que lo reduce.
- **¿Qué significa semi-gradiente?:** En el cálculo convencional, si una meta cambia a medida que te mueves, tendrías que calcular cómo tus movimientos afectan a la propia meta. Pero aquí entra la genialidad de la Red Target: sus perillas están congeladas en el tiempo. La tratamos simplemente como un cartel con un número fijo anotado en la pared. El optimizador solo mueve las perillas de la Red Policy para alcanzar esa cifra estática, sin preocuparse por derivar ni alterar la diana durante el disparo.

6.4 Cálculo Numérico Paso a Paso: Los 10 Pasos de DQN en OptiBot

Para ver cómo opera la matemática con absoluta nitidez, sigamos una vivencia real de OptiBot en el almacén continuo, desglosando los 10 pasos con números exactos.

Consideremos el entorno continuo del almacén de 3×3 metros con factor de descuento $\gamma = 0.95$ y tasa de aprendizaje $\eta = 0.001$.

Pasos 1 y 2: Arquitectura y pesos iniciales de las dos redes

Se inicializa la **Red Policy** ($\boldsymbol{\theta}_{\text{policy}}$) con perillas aleatorias y se clona para crear la **Red Target** ($\boldsymbol{\theta}_{\text{target}}$). Ambas redes tienen 4 entradas continuas y 4 salidas cardinales:

$$\mathcal{A} = \{0 : \text{Norte } (\uparrow),\ 1 : \text{Sur } (\downarrow),\ 2 : \text{Este } (\rightarrow),\ 3 : \text{Oeste } (\leftarrow)\}$$

Paso 3, 3a y 3b: Medición sensorial, Álbum de Recuerdos y Muestreo

OptiBot se ubica en el Muelle de Carga (S_1) y mide su vector continuo:

$$\mathbf{s} = [0.55 \text{ m},\ 2.45 \text{ m},\ +1.95 \text{ m},\ -1.95 \text{ m}]^T$$

El robot ejecuta la acción $a = 2$ (*Este* $\rightarrow$, iniciando el avance hacia el pasillo despejado). El entorno físico devuelve:

- Recompensa inmediata recibida: $r = -1.00$ (costo por tiempo de paso).
- Nuevo estado continuo alcanzado: $\mathbf{s}' = [1.02 \text{ m},\ 2.44 \text{ m},\ +1.48 \text{ m},\ -1.94 \text{ m}]^T$.
- Fin de episodio: terminal $=$ Falso (0).

En el **Paso 3a**, la experiencia $(\mathbf{s}, a = 2, r = -1.00, \mathbf{s}', \text{terminal} = 0)$ se guarda en el Experience Replay. En el **Paso 3b**, el muestreador extrae esta experiencia para entrenar.

Paso 4: Propagación en la Red Policy y predicción de salida

El vector de estado $\mathbf{s}$ entra por la capa de entrada de la Red Policy. Sus neuronas de salida emiten los siguientes valores de recompensa esperada:

Acción	0 (Norte $\uparrow$)	1 (Sur $\downarrow$)	2 (Este $\rightarrow$)	3 (Oeste $\leftarrow$)
$Q(\mathbf{s}, a; \boldsymbol{\theta}_{\text{policy}})$	2.80	1.90	$\mathbf{3.10}$	1.40

La predicción que la red tenía para la acción ejecutada ($a = 2$, Este) era $Q(\mathbf{s}, a = 2; \boldsymbol{\theta}_{\text{policy}}) = \mathbf{3.100}$.

Paso 5: Propagación en la Red Target con el mismo vector de estado

El vector de estado $\mathbf{s}$ entra a la Red Target congelada, emitiendo la plantilla base de salida:

$$Q(\mathbf{s}, \cdot; \boldsymbol{\theta}_{\text{target}}) = [2.80,\ 1.90,\ 3.10,\ 1.40]$$

Paso 6: Cálculo del valor objetivo de Bellman (Q_{target})

Para calcular cuánto valía realmente esa maniobra, introducimos el **siguiente estado alcanzado** $\mathbf{s}'$ en la Red Target congelada:

Acción futura	$a' = 0$ (Norte)	$a' = 1$ (Sur)	$a' = 2$ (Este)	$a' = 3$ (Oeste)
$Q(\mathbf{s}', a'; \boldsymbol{\theta}_{\text{target}})$	4.20	$\mathbf{6.50}$	5.80	1.10

La mejor opción futura predicha por la Red Target congelada es ir hacia el Sur ($a' = 1$) con un valor de:

$$\max_{a' \in \mathcal{A}} Q(\mathbf{s}', a'; \boldsymbol{\theta}_{\text{target}}) = \mathbf{6.50}$$

Como el estado no es terminal, aplicamos la Ecuación (6.7):

$$Q_{\text{target}} = r + \gamma \max_{a'} Q(\mathbf{s}', a'; \boldsymbol{\theta}_{\text{target}}) = -1.00 + (0.95 \times 6.50) = -1.00 + 6.175 = \mathbf{5.175}$$

Paso 7: Establecimiento del Vector Target

Tomamos el vector de salida que emitió la Red Target en el Paso 5, $[2.80, 1.90, 3.10, 1.40]$, y **reemplazamos la casilla de la acción tomada ($a = 2$, Este)** por el valor de Bellman $Q_{\text{target}} = 5.175$:

$$\mathbf{y}_{\text{target}} = \begin{bmatrix} 2.80 \\ 1.90 \\ \mathbf{5.175} \\ 1.40 \end{bmatrix}$$

Las acciones $0, 1$ y 3 conservan sus valores originales intactos.

Paso 8: Entrenamiento de la Red Policy con Descenso de Gradiente

Calculamos la discrepancia cuadrática entre lo que la Red Policy predijo y el vector Target:

$$\begin{aligned} \mathcal{L} &= (2.80 - 2.80)^2 + (1.90 - 1.90)^2 + (\mathbf{5.175} - \mathbf{3.100})^2 + (1.40 - 1.40)^2 \\ &= 0 + 0 + (+2.075)^2 + 0 = (2.075)^2 = \mathbf{4.306} \end{aligned}$$

Interpretación clara para dummies:

- Para Norte, Sur y Oeste el error es exactamente 0. Sus perillas no sufrirán ninguna alteración espuria.
- Para el Este ($a = 2$), la sorpresa temporal es $\delta = 5.175 - 3.100 = +\mathbf{2.075}$. ¡Una maravillosa noticia! La acción Este resultó ser $+2.075$ puntos mejor de lo que la Red Policy creía.
- El algoritmo retropropaga este error de 4.306 y gira las perillas de la Red Policy para elevar su estimación del Este hacia el blanco de 5.175.
- La Red Target continúa $100\,\%$ inmóvil, ofreciendo una diana fija y libre de temblores.

Paso 9: Bucle continuo de exploración

OptiBot continúa navegando en el almacén continuo, acumulando vivencias en su Experience Replay y entrenando la Red Policy paso tras paso.

Paso 10: Sincronización periódica de las redes

Tras cumplirse $C = 4$ iteraciones de entrenamiento, las perillas aprendidas en la Red Policy se transfieren a la Red Target:

$$\boldsymbol{\theta}_{\text{target}} \leftarrow \boldsymbol{\theta}_{\text{policy}}$$

La diana da un salto de calidad reflejando la nueva maestría de OptiBot y vuelve a congelarse para la siguiente ronda de aprendizaje.

Preguntas Frecuentes para Dummies sobre Aprendizaje por Refuerzo Profundo:

1. **¿Cómo explora el robot si la red neuronal siempre arroja números continuos?** Seguimos aplicando la clásica regla ε-greedy: con probabilidad ε, OptiBot ignora la red por completo y elige una de las cuatro acciones lanzando una moneda al azar. Con probabilidad $1 - \varepsilon$, explota seleccionando la neurona de salida con el valor más alto ($\arg\max$).

2. **¿Qué ocurre cuando el Experience Replay se llena por completo?** Funciona como una cola circular FIFO de tamaño fijo (por ejemplo, $50{,}000$ vivencias). Cuando el álbum se satura, cada nueva experiencia expulsa automáticamente al recuerdo más antiguo para dejar espacio libre.

3. **¿Por qué no usamos redes neuronales para todo desde el primer día?** Porque para problemas sencillos con pocos estados discretos (como la cuadrícula simple de 3×3 casillas), las tablas tradicionales convergen con certeza matemática garantizada, se calculan en microsegundos y no requieren calibrar miles de perillas ni lidiar con aproximaciones estadísticas. Las redes neuronales son la artillería pesada que desplegamos cuando el mundo se vuelve continuo o infinitamente complejo.

6.5 Implementación Práctica: OptiBot DQN en el Cuaderno Interactivo

Para experimentar en vivo con el entrenamiento de redes neuronales profundas para control por refuerzo, el código completo y ejecutable se encuentra disponible en el cuaderno interactivo del curso:

`notebooks/cap05_rl_profundo.ipynb`

En este cuaderno interactivo se implementa el entorno continuo del almacén de OptiBot, la red neuronal de aproximación de valor, el búfer de repetición de experiencias (para descorrelacionar las vivencias temporales) y el agente completo integrando la arquitectura dual de Deep Q-Learning (DQN). El cuaderno incluye bucles de entrenamiento en coordenadas continuas, seguimiento visual de la curva de pérdida (MSE) y la animación gráfica de la trayectoria continua recorrida por el robot esquivando la estantería central hasta alcanzar con éxito la Bahía de Entrega.

6.6 Comparativa Global: De las Tablas a las Redes Neuronales

La Tabla 6.2 resume la transición paradigmática recorrida a lo largo de este curso, contrastando las dos grandes eras del Aprendizaje por Refuerzo:

Dimensión	RL Tradicional (Q-Learning)	RL Profundo (DQN)
Representación	Matriz o tabla discreta $Q(s, a)$.	Red Policy paramétrica $Q(\mathbf{s}, a; \boldsymbol{\theta}_{\text{policy}})$.
Espacio de Estados	Finito y reducido (pocos estados discretos).	Continuo, sensorial y métrico ($\mathbb{R}^4$, distancias relativas).
Generalización	Nula: cada celda (s, a) debe visitarse de forma individual.	Total: interpola comportamientos óptimos para estados jamás observados.
Garantías Teóricas	Convergencia asintótica garantizada con probabilidad 1 (Watkins).	Convergencia empírica estabilizada mediante arquitectura dual.
Estabilizadores Clave	Tasa de aprendizaje decreciente α_t y política ε-greedy.	Experience Replay (cola FIFO) y Red Target congelada ($\boldsymbol{\theta}_{\text{target}}$).

Cuadro 6.2: Comparativa estructural entre el Aprendizaje por Refuerzo Tradicional y Profundo.

Conclusión y Perspectivas Futuras: A lo largo de este manual, hemos construido de forma progresiva, rigurosa y transparente el andamiaje completo del Aprendizaje por Refuerzo:

- Partimos de la cuantificación de la incertidumbre probabilística y los bandidos multi-brazo.
- Formalizamos las transiciones dinámicas mediante Cadenas y Procesos de Decisión de Markov, deduciendo analíticamente las Ecuaciones de Bellman.
- Desarrollamos el control libre de modelo mediante Diferencia Temporal y Q-Learning tradicional.
- Finalmente, conquistamos los espacios continuos mediante el Aprendizaje por Refuerzo Profundo y las redes DQN.

Con estas bases matemáticas y computacionales, el estudiante se encuentra plenamente capacitado para abordar las fronteras avanzadas del aprendizaje autónomo, tales como los métodos de Gradiente de Política (*Policy Gradients*, PPO, SAC) y los sistemas modernos de alineamiento inteligente.